



PDF Download
3664607.pdf
27 February 2026
Total Citations: 4
Total Downloads: 750

Latest updates: <https://dl.acm.org/doi/10.1145/3664607>

RESEARCH-ARTICLE

Mobile Application Online Cross-Project Just-in-Time Software Defect Prediction Framework

SIYU JIANG, Guangdong University of Foreign Studies, Guangzhou, Guangdong, China

ZHENHANG HE, Guangdong University of Foreign Studies, Guangzhou, Guangdong, China

YUWEN CHEN, Guangdong University of Foreign Studies, Guangzhou, Guangdong, China

MINGRONG ZHANG, Guangzhou City University of Technology, Guangzhou, Guangdong, China

LE MA, Guangzhou City University of Technology, Guangzhou, Guangdong, China

Open Access Support provided by:

Guangzhou City University of Technology

Guangdong University of Foreign Studies

Published: 27 June 2024

Online AM: 14 May 2024

Accepted: 15 April 2024

Revised: 06 March 2024

Received: 20 November 2023

[Citation in BibTeX format](#)

Mobile Application Online Cross-Project Just-in-Time Software Defect Prediction Framework

SIYU JIANG, School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou, China

ZHENHANG HE, School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou, China

YUWEN CHEN, School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou, China

MINGRONG ZHANG, Guangzhou City University of Technology, Guangzhou, China

LE MA, Guangzhou City University of Technology, Guangzhou, China

As mobile applications evolve rapidly, their fast iterative update nature leads to an increase in software defects. Just-In-Time Software Defect Prediction (JIT-SDP) offers immediate feedback on code changes. For new applications without historical data, researchers have proposed Cross-Project JIT-SDP (CP JIT-SDP). Existing CP JIT-SDP approaches are designed for offline scenarios where target data is available in advance. However, target data in real-world applications usually arrives online in a streaming manner, making online CP JIT-SDP face cross-project distribution differences and target project data concept drift challenges in online scenarios. These challenges often co-exist during application development, and their interactions cause model performance to degrade. To address these issues, we propose an online CP JIT-SDP framework called COTL. Specifically, COTL consists of two stages: offline and online. In the offline stage, the cross-domain structure preserving projection algorithm is used to reduce the cross-project distribution differences. In the online stage, target data arrives sequentially over time. By reducing the differences in marginal and conditional distributions between offline and online data for target project, concept drift is mitigated and classifier weights are updated online. Experimental results on 15 mobile application benchmark datasets show that COTL outperforms 13 benchmark methods on four performance metrics.

CCS Concepts: • **Software and its engineering** → **Software defect analysis; Maintaining software;**

Additional Key Words and Phrases: Online transfer learning, mobile applications bug prediction, cross-project just-in-time software defect prediction, concept drift

ACM Reference Format:

Siyu Jiang, Zhenhang He, Yuwen Chen, Mingrong Zhang, and Le Ma. 2024. Mobile Application Online Cross-Project Just-in-Time Software Defect Prediction Framework. *ACM Trans. Softw. Eng. Methodol.* 33, 6, Article 157 (June 2024), 31 pages. <https://doi.org/10.1145/3664607>

Authors' Contact Information: Siyu Jiang, School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou, Guangdong, China; e-mail: jiangsiyu@gdufs.edu.cn; Zhenhang He, School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou, Guangdong, China; e-mail: 20221050030@gdufs.edu.cn; Yuwen Chen, School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou, Guangdong, China; e-mail: 20221050011@gdufs.edu.cn; Mingrong Zhang, Guangzhou City University of Technology, Guangzhou, Guangdong, China; e-mail: 153155269@qq.com; Le Ma, Guangzhou City University of Technology, Guangzhou, Guangdong, China; e-mail: 80796487@qq.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1049-331X/2024/06-ART157

<https://doi.org/10.1145/3664607>

1 INTRODUCTION

In recent years, the use of mobile applications has experienced rapid growth, becoming an essential component of our daily lives. Through mobile applications, we can stay closely connected and solve various practical problems [15, 24]. Moreover, the frequent iteration and updates of these applications provide users with the latest features and experiences, meeting the diverse needs of different user groups. However, as mobile applications continue to grow in both scale and complexity, software updates unavoidably brings about software defects [36, 47]. These defects may affect the normal operation of applications, leading to unpleasant user experiences, and even compromising user data security and privacy protection. Therefore, early detection and resolution of these defects are crucial. To this end, scholars proposed the **software defect prediction (SDP)** techniques [10, 13, 21, 50], utilizing machine learning methods for specific code regions to predict the defects. This technology can effectively improve software quality assurance, reduce manual work and save time.

The frequent updating nature of mobile applications has not been recognized in many previous SDP studies [14, 16, 26, 31, 41]. These studies focused on predicting defects in file-level or class-level code segments, and they required all file-level or class-level code to be combined and re-predicted, and could not predict the presence of defects in new code segments that were updated in real-time. In light of this, researchers have introduced the **just-in-time software defect prediction (JIT-SDP)** method [17, 18, 27, 35], which strives to promptly identify potential defect risks at the moment of code commitment. After committing the code involved in this modification as a new code instance, the JIT-SDP method would predict the likelihood of having a defective code change. By providing feedback at the commit level, JIT-SDP affords developers with immediate defect detection and remediation, thereby ensuring the stability and reliability of applications. Catolino et al. [5] were the pioneers in applying JIT-SDP to mobile applications. They conducted an analysis of code commit instances and developed a classification model to predict defects in mobile applications. This research not only demonstrates the practical application of JIT-SDP in mobile applications, but also lays the foundation for future research.

As new mobile applications continue to be developed to meet the needs of the public, a common challenge arises: lack of historical data required to perform within-project JIT-SDP. To address this issue, previous research proposed **cross-project JIT-SDP (CP JIT-SDP)** [6, 43], which utilizes a classifier trained with historical data from another project (source project) to predict defects in another project (target project). There are differences in data distribution between projects, such as differences in project size, functionality, and coding habits. As a result, it is not possible to directly apply to the target project what was learned in the source project. To overcome these distributional differences, transfer learning [29] is employed to transfer their data from both projects to a new feature space which has similar distributions after the transformation. The primary purpose of this is to enhance the predictive capability of the classifier and ultimately improve its performance. One such solution is proposed by Cheng et al. [6], who introduced the KAL method that combines kernel method and adversarial learning for the CP JIT-SDP task. Initially, they mapped the raw data to a high-dimensional space using kernel principal component analysis. Subsequently, they extracted common features from this space using adversarial learning techniques. Following a similar pursuit of advancing JIT-SDP methods, Xu et al. [43] proposed a method called **CDFE (cross-triple deep feature embedding)**, which aims to learn high-level feature representations cross projects. The method incorporated in the deep neural network framework a state-of-the-art cross-triple loss function.

Existing CP JIT-SDP methods typically use an offline learning approach. This involves collecting target project data beforehand and training a classifier on source project offline data. The classifiers are then utilized to perform defect prediction on target project offline data. This offline approach

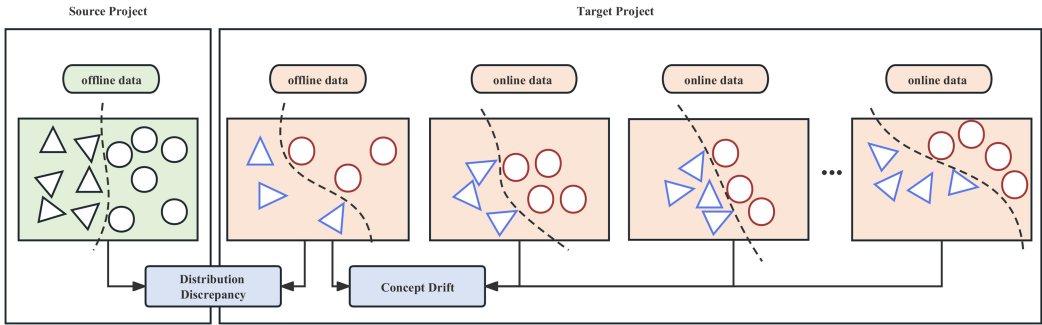


Fig. 1. In the scenario of online CP JIT-SDP, data from target project arrives in a streaming manner. In this case, there not only exist distribution differences between offline data of source project and target project, but also the concept drift between upcoming online data and collected offline data within target project.

lacks the ability to incorporate the target project's new data and update the classifier, and therefore does not enable online learning. In reality, CP JIT-SDP operates as an online learning problem [1, 3, 32, 33] since data from the target project accumulates over time, thus creating a stream of data to update the model in a timely manner. At the same time, the phenomenon that the data distribution changes over time when certain environmental factors for the generation of streaming data change is called the concept drift, and this phenomenon also affects the CP JIT-SDP [2, 3, 25]. This is because as a software project evolving, software features may be adapted or improved at different development stages in response to changes in time, market and user requirements, which in turn leading to changes in the defect generation process. Therefore, classifiers trained only on data from a specific period may not be applicable to subsequent periods. During the actual development of mobile applications, cross-project distributional differences and conceptual drift often coexist, and their interactions may exacerbate the degradation of predictive model performance. As shown in Figure 1, discernible disparities in distribution, denoting incongruities in defect and non-defect features, emerge within the offline data gathered from both the source and target projects. Furthermore, there exists a concept drift between the offline and online data of target project itself. The feature distribution of target project data also varies over time, and some commit instances of the target project data may not fully represent the overall feature distribution of the data. Additionally, offline data for target projects may be distributed differently with online data. Such inconsistencies in the distribution of data features as described above result in the classification boundaries learned by the original model no longer being applicable, thus affecting the prediction performance. Figure 2 further shows examples of commits in real-world scenarios of mobile application development. Figure 2(a) shows the latest set of commits for the Android mobile application Delta, where the developer committed eight code changes in two days, reflecting the high frequency of updates that is common in real-world mobile application development and demonstrating the importance of online scenarios. Figure 2(b) shows a recent commit for the purpose of "updating local help". This commit involved changes to seven files, including the addition of 192 lines of code and the deletion of 466 lines of code. Such code additions and deletions may not only introduce new defects or eliminate old defect patterns, but may also change the internal structure of the application, thereby affecting the distribution of defects. In the environment of frequent updates over time, each code modification may cause a change in the defect distribution, leading to concept drift and thus biasing the model predictions. Consequently, it is important to explore an online learning CP JIT-SDP approach to detect commits and better adapt it to real-world application development scenarios.

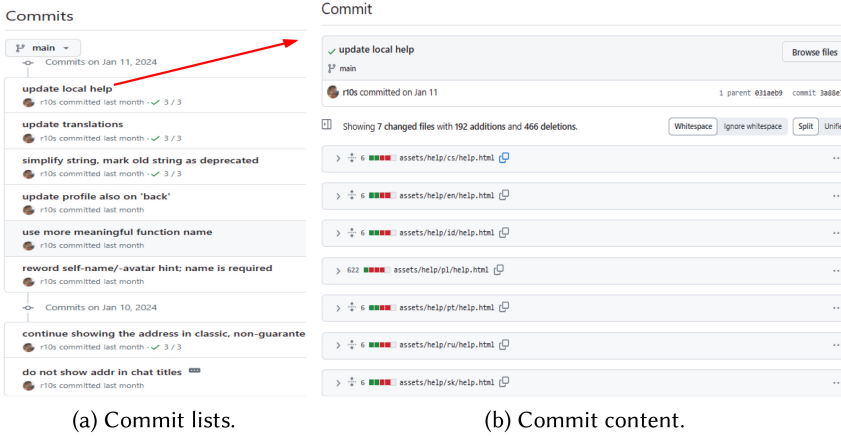


Fig. 2. An example of commits in a real-world scenario of mobile application development.

To address the aforementioned issues, we propose an online CP JIT-SDP framework called COTL, which combines the advantages of cross-domain structure-preserving projection and online transfer learning. This framework is distinctively designed to accommodate the variances in marginal and conditional distributions present in the streaming data of the target project, thereby aiming to boost classifier efficiency and accuracy. The COTL framework includes two stages: an initial offline stage and a subsequent online stage. In the offline stage, a cross-domain structure-preserving projection is employed to map both the source and target projects into a common subspace, with the primary goal of mitigating differences within the distributions of these two projects. Specifically, we employ the cross-domain structure-preserving projection to learn specific projections of source project and a limited number of target project data. Once the projection matrix is obtained, we train the classifier in the common subspace to ensure class consistency and sufficient alignment of data distributions. In the online stage, we adapt the streaming target project data to the classifier trained in the common subspace by addressing concept drift. More specifically, we continuously input target project data in an online manner and train online classifiers in the common subspace, updating the weights of the classifiers. Additionally, we reduce the impact of concept drift through minimizing the difference between the marginal and conditional distributions of the target project.

The main contributions of this paper are highlighted as follows:

- We propose a novel online CP JIT-SDP framework called COTL, which integrates cross-domain structure preserving projection and online transfer learning. The framework not only bridges the distribution difference between source and target projects, but also dynamically updates the model in online scenarios where target project data arrives over time, making it more applicable to real-world situations.
- We dynamically reduce the differences in marginal and conditional distributions between target project data in the online stage by utilizing a dynamic data distribution adaptation mechanism, thereby mitigating the concept drift problem and improving the performance of defect prediction.
- We conducted comprehensive experiments on a benchmark dataset comprising 15 mobile applications, encompassing a total of 210 cross-project pairs. The results from the experimentation and statistical analysis affirm the efficacy of the COTL framework proposed in our study.

The article structure is as follows: Section 2 provides related work. Section 3 presents the proposed framework. Section 4 describes the setup of the experimental evaluation. Section 5 evaluates the effectiveness of the proposed framework. Section 6 discusses the threats to validity. Finally, Section 7 summarizes the findings of this study.

2 RELATED WORK

2.1 Just-in-Time Software Defect Prediction

JIT-SDP is an active research area that aims to predict in real-time whether defects exist in code changes committed by developers. This enables rapid feedback to be provided to developers before code integration. Initial work by Kamei et al. [18] introduced 14 change metrics to describe software changes and validated logistic regression models on 11 open-source and commercial projects. Since then, researchers have explored more sophisticated machine learning techniques. Yang et al. [44] were the first to apply deep learning, using deep belief networks to learn expressive feature representations. On the same dataset, their deeper method outperformed that of Kamei et al. [18]. Subsequently, they introduced a new two-layer ensemble method, TLEL, which combines ensemble learning with the random forest method. TLEL achieved better performance, demonstrating the promise of ensemble techniques.

Due to frequent updates of mobile applications, timely feedback is particularly important. Catolino [4] conducted an initial study on five mobile applications using logistic regression, finding suboptimal performance and a need for better models. More recently, Zhao et al. [46] developed a combined model called KPIDL, utilizing feature learning and cost-sensitive loss functions to address class imbalance. Experiments on 15 mobile applications showed superior performance over 25 baselines. They also proposed a simplified deep forest approach [47] that integrates traditional tree ensembles, which outperformed 16 baselines on 10 mobile applications.

The aforementioned studies were conducted within projects, but for new projects, defect prediction models cannot be constructed owing to the lack of sufficient historical data. Therefore, in this study, we primarily focus on CP JIT-SDP research.

2.2 Cross-Project Just-In-Time Software Defect Prediction

CP JIT-SDP is a method trained with source project data to predict the presence of defects in target projects that lack historical data. Kamei et al. [17] investigated 11 open-source projects and found better performance by carefully selecting cross-project training data compared to within-project models. This demonstrated the feasibility of CP JIT-SDP, spurring further research. A variety of techniques have been explored to improve cross-project generalization. For instance, Zhu et al. [51] developed a deep learning model called DAECNN-JDP, which uses denoising autoencoders and **convolutional neural networks (CNNs)**. The autoencoders learn robust feature representations, while the CNNs capture spatial relationships. In experiments on six large open-source projects, DAECNN-JDP outperformed 11 baselines.

For mobile applications, which frequently release updates, Catolino et al. [5] were the first to introduce the CP JIT-SDP model in this context. They verified the effectiveness of traditional classification models and ensemble techniques in CP JIT-SDP, with their experiments involving 14 mobile applications. The outcomes revealed that the plain Bayes classifier yielded the most favorable results, surpassing other classifiers and some integration techniques. Recent work has focused on using deep learning to learn transferable feature representations. For example, Cheng et al. [6] developed a model named KAL, which uses kernel-based principal component analysis for dimensionality reduction and adversarial learning to reduce cross-project distribution divergence. KAL achieved strong performance on 182 cross-project pairs from 14 mobile applications, outperforming 12 baselines, including instance selection, classifier ensembles, and

KAL variants without KPCA or adversarial learning. Xu et al. [43] proposed a method named CDFE, a cross-domain metric learning method using cross-triplet loss, which was evaluated with 19 mobile applications. In comparison experiments, CDFE outperformed 14 other methods. Furthermore, Huang et al. [12] introduced MTL-DNN, a method utilizing multi-task learning to address the challenge of limited source and target project label training data. MTL-DNN, which learns shared layers to capture common features across multiple tasks and task-specific layers for each task, demonstrated superior performance on 15 mobile applications, significantly outperforming single-task and other machine learning methods.

Although CP JIT-SDP has achieved good results, the aforementioned studies are based on offline scenarios where the model is not updated with new data and do not consider the realistic issue of target project data arriving over time in online learning scenarios.

2.3 Online Transfer Learning

An offline scenario is usually the basis for traditional transfer learning methods, which assume that complete source and target domain data can be obtained in advance. In this setup, the source domain data is used to train a model, which is then applied to a task in the target domain. However, online transfer learning differs from traditional transfer learning in that it assumes that labeled source data can be obtained beforehand, while target data will gradually arrive over time. This assumption is more in line with actual scenarios, necessitating a model that can dynamically adapt to changes in the target domain. Researchers have proposed various algorithmic frameworks to address this challenge.

Early research focused on building frameworks for incremental learning. Zhao et al. [48] pioneered the concept and algorithmic framework of **online transfer learning (OTL)**, which integrates dynamically weighted source-domain classifiers with subsequent target-domain classifiers. Wu et al. [40] expanded on this framework by proposing algorithms that utilize data from multiple source domains simultaneously to improve classification performance in the target domain, facilitating knowledge transfer from multiple sources to a single target. Wu et al. [39] further extended their approach to heterogeneous source domains, proposing an algorithm to dynamically adjust the weights of these domains. Subsequently, researchers extended online transfer learning to more complex scenarios. For multi-class classification tasks, Kang et al. [19] introduced the OTLAMC algorithm, employing a novel loss function, model representation, and parameter updating method for online transfer in multi-class scenarios. Kang et al. [20] conducted an in-depth study of the intrinsic relationship between multi-classes using the OTLAMC algorithm, utilizing an extended hinge loss function. Additionally, Du et al. [9] proposed a homogeneous online transfer learning algorithm to address the distribution differences between source and target domains. This algorithm employs a hedging strategy to reduce the marginal and conditional distribution differences online, thereby enhancing the model's predictive performance.

Inspired by these developments, in this study, we incorporate the concept of online transfer learning and propose a novel online CP JIT-SDP method called COTL, designed to manage situations where source domain data is available offline while target domain data arrives sequentially.

3 APPROACH

3.1 Problem Definition

In CP JIT-SDP, we face a homogenous binary classification problem. The given data consists of source project data denoted as $X_s \in R^{m \times n_s}$ and target project data denoted as $X_t \in R^{m \times n_t}$, where n_s and n_t denote the sample numbers of source and target projects, the m denotes the dimensionality of data features, respectively. In practical applications, we typically only have access to labels

Table 1. Notation and Description in this Study

Notation	Description
X_s	labeled source project offline data
X_t	labeled target project offline data
X_{t_i}	unlabeled target project online data
Y_s	the label of source project data
Y_t	the label of target project data
n_s	the number of samples in source project
n_t	the number of samples in target project
V_s	projection matrices of source project data
V_t	projection matrices of target project data
Z_s	common subspace data of source project data
Z_t	common subspace data of target project data
f_s	the classifier of source project
f_t	the classifier of target project
w_t	the weight vector of the classifier at round t
λ_t	the weight of source classifier at round t
β_t	the weight of target classifier at round t
T_w	time window

Y_s of source project, while labels Y_t of target project are unknown. In an online setting, target data arrives with time changes in streaming manner, and X_{t_i} denotes the incoming data at time i . The main goal of this study is to learn a model using labeled source data and limited labeled target data that enables prediction of unlabeled data X_{t_i} from target projects and can be updated based on feedback from X_{t_i} . Key notation used throughout this paper is defined in the Table 1.

In this study, we propose the framework called COTL, which consists of two different stages as shown in Figure 3. First, in the offline stage, the projection matrices of the source and target projects are obtained by the cross-domain structure-preserving projection. These matrices are used to transform the data into a common subspace, thus effectively reducing the distributional differences. Then, in the online stage, concept drift is reduced by reducing the marginal and conditional distributional differences of target project data, followed by updating the classifier weights to produce the final classification results.

3.2 Offline Stage

In the offline stage, there are available labeled source data and limited labeled target data. At this point, a significant challenge is the existence of distributional differences between the source and target data. If no measures are taken to minimize these distributional differences when training the classifier on source data, then when applied to target data, the performance of the classifier drops significantly. For this problem, we employ the cross-domain structure preserving projection algorithm, the improved version of the locality preserving projections algorithm [11, 37]. The algorithm minimizes the effect of distributional differences by mapping the original data of source and target projects into a common subspace. Specifically, the algorithm introduces three core adjacency matrices, W_s , W_t , and W_c , which represent the similarity between source, target, and cross-domain data, respectively. These matrices are constructed based on the matching relationship of data labels. By learning the projection matrices V_s and V_t for the source and target projects, the algorithm can keep the original data structure of the projected data in the subspace while keeping the same samples from different projects close to each other.

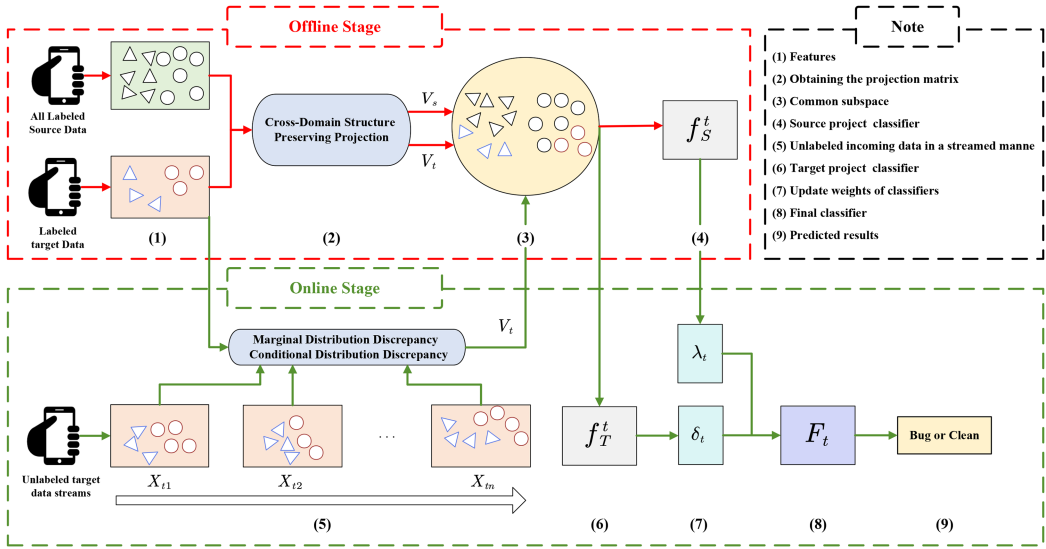


Fig. 3. The framework we propose consists of two stages. In the offline stage, we employ cross-domain structure-preserving projection algorithm to map the data from source and target projects to a common subspace, reducing the distribution discrepancy between them. In the online stage, we reduce concept drift by minimizing the differences in marginal and conditional distributions among target project data, and update the weights of the classifier to obtain the final classification results.

The internal similarity of the source project data is represented by the adjacency matrix $W_s \in R^{n_s \times n_s}$. Two data x_i^s and x_j^s are considered adjacent if they have the same label, i.e., $y_i^s = y_j^s$. It is defined as follows:

$$W_{ij}^s = \begin{cases} 1, & \text{if } y_i^s = y_j^s \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

The internal similarity of the target project data is represented by the adjacency matrix $W_t \in R^{n_t \times n_t}$. Similarly, x_i^t and x_j^t are considered adjacent when they have the same label ($y_i^t = y_j^t$). It is defined as follows:

$$W_{ij}^t = \begin{cases} 1, & \text{if } y_i^t = y_j^t \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

Cross-domain similarity is represented by the adjacency matrix $W_c \in R^{n_s \times n_t}$. The source data x_i^s and the target data x_j^t are considered similar when their labels match ($y_i^s = y_j^t$). It is defined as follows:

$$W_{ij}^c = \begin{cases} 1, & \text{if } y_i^s = y_j^t \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

After constructing the three core adjacency matrices, the optimization objective can be expressed as:

$$\min_{V_s, V_t} \left(\sum_{i,j}^{n_s} \|V_s^T x_i^s - V_s^T x_j^s\|_2^2 W_{ij}^s + \sum_i^{n_s} \sum_j^{n_t} \|V_s^T x_i^s - V_t^T x_j^t\|_2^2 W_{ij}^c + \sum_{i,j}^{n_t} \|V_t^T x_i^t - V_t^T x_j^t\|_2^2 W_{ij}^t \right) \quad (4)$$

where V_s^T is the transpose of V_s and V_t^T is the transpose of V_t . The objective function represented in Equation (4) can be expanded using a binary specification, which subsequently simplified as the

following:

$$\min_{V_s, V_t} \left(\text{tr} \left(X_s^T V_s V_s^T X_t L_s \right) + \text{tr} \left(X_t^T V_t V_t^T X_t L_t \right) - \text{tr} \left(X_s^T V_s V_t^T X_t W_c^T \right) \right) \quad (5)$$

where $L_s = D_s - W_s + \frac{1}{2}D_{cs}$; $D_s \in R^{n_s \times n_s}$ and $D_{cs} \in R^{n_s \times n_s}$ is the diagonal matrix with $D_{ii}^s = \sum_j^{n_s} W_{ij}^s$; $D_{ii}^{cs} = \sum_j^{n_t} W_{ij}^c$. $L_t = D_t - W_t + \frac{1}{2}D_{ct}$; $D_t \in R^{n_t \times n_t}$ and $D_{ct} \in R^{n_t \times n_t}$ is the diagonal matrix with $D_{jj}^t = \sum_i^{n_t} W_{ij}^t$; $D_{jj}^{ct} = \sum_i^{n_s} W_{ij}^c$.

Specifically, each term in Equation (4) uses a binomial expansion, which is then converted to matrix multiplication and matrix trace forms. For the first term in Equation (4), it can be re-expressed as the following:

$$\begin{aligned} \sum_{i,j}^{n_s} \|V_s^T x_i^s - V_s^T x_j^s\|_2^2 W_{ij}^s &= \sum_{i,j}^{n_s} \left(x_i^{sT} V_s V_s^T x_i^s - 2x_i^{sT} V_s V_s^T x_j^s + x_j^{sT} V_s V_s^T x_j^s \right) W_{ij}^s \\ &= 2 \sum_i^{n_s} x_i^{sT} V_s V_s^T x_i^s D_{ii}^s - 2 \sum_{i,j}^{n_s} x_i^{sT} V_s V_s^T x_j^s W_{ij}^s \\ &= 2 \text{tr} \left(X_s^T V_s V_s^T X_s D_s \right) - 2 \text{tr} \left(X_s^T V_s V_s^T X_s W_s \right) \end{aligned} \quad (6)$$

For the second term in Equation (4), it can be re-expressed as the following:

$$\begin{aligned} \sum_i^{n_s} \sum_j^{n_t} \|V_s^T x_i^s - V_t^T x_j^t\|_2^2 W_{ij}^c &= \sum_i^{n_s} \sum_j^{n_t} \left(x_i^{sT} V_s V_s^T x_i^s - 2x_i^{sT} V_s V_t^T x_j^t + x_j^{tT} V_t V_t^T x_j^t \right) W_{ij}^c \\ &= \sum_i^{n_s} x_i^{sT} V_s V_s^T x_i^s D_{ii}^{cs} - 2 \sum_i^{n_s} \sum_j^{n_t} x_i^{sT} V_s V_t^T x_j^t W_{ij}^c + \sum_j^{n_t} x_j^{tT} V_t V_t^T x_j^t D_{jj}^{ct} \\ &= \text{tr} \left(X_s^T V_s V_s^T X_s D_{cs} \right) - 2 \text{tr} \left(X_s^T V_s V_t^T X_t W_c^T \right) + \text{tr} \left(X_t^T V_t V_t^T X_t D_c^T \right) \end{aligned} \quad (7)$$

Similarly, for the third term in Equation (4), it can be re-expressed as the following:

$$\sum_{i,j}^{n_t} \|V_t^T x_i^t - V_t^T x_j^t\|_2^2 W_{ij}^t = 2 \text{tr} \left(X_t^T V_t V_t^T X_t D_t \right) - 2 \text{tr} \left(X_t^T V_t V_t^T X_t W_t \right) \quad (8)$$

Substituting each term in Equation (4) gives the simplified Equation (5). Mathematically, the Equation (5) can be converted equivalently:

$$\max_{V_s, V_t} \frac{\text{tr}(X_s^T V_s V_t^T X_t W_c^T)}{\text{tr}(X_s^T V_s V_s^T X_s L_s) + \text{tr}(X_t^T V_t V_t^T X_t L_t)} \quad (9)$$

The generalized eigenvalue decomposition method can be used to solve the Equation (9). The optimal projection matrix $V = \begin{bmatrix} V_s \\ V_t \end{bmatrix}$ can be obtained by calculating the corresponding eigenvalues and eigenvectors of the matrix, checking the eigenvalues in descending order, and finding the d eigenvectors corresponding to the largest eigenvalues of d :

$$AV = (B + \alpha I)V\Lambda \quad (10)$$

where α is the regularization parameter, $I \in R^{(n_s \times n_t) \times (n_s \times n_t)}$ is the identity matrix; Λ represents the diagonal eigenvalue matrix and:

$$A = \begin{bmatrix} 0 & X_s W_c X_t^T \\ X_t W_c^T X_s^T & 0 \end{bmatrix} \quad (11)$$

$$B = \begin{bmatrix} X_s L_s X_s^T & 0 \\ 0 & X_t L_t X_t^T \end{bmatrix} \quad (12)$$

Let $P_s = X_s L_s X_s^T$, $P_t = X_t L_t X_t^T$ and $P_c = X_s W_c X_t^T$. Combining Equation (11) and Equation (12), we can easily obtain the following generalized eigenvalue problem:

$$\begin{bmatrix} 0 & P_c \\ P_c^T & 0 \end{bmatrix} \begin{bmatrix} V_s \\ V_t \end{bmatrix} = \begin{bmatrix} P_s & 0 \\ 0 & P_t \end{bmatrix} \begin{bmatrix} V_s \\ V_t \end{bmatrix} \Lambda \quad (13)$$

Finally, it can calculate the optimal projection matrices V_s and V_t to realize the knowledge transfer between source and target projects.

Once the projection matrices V_s and V_t are learned, all labeled samples can be projected into the learned common subspace with $Z_s = V_s^T x_i^s$ and $Z_t = V_t^T x_i^t$. Afterwards, source and target project data are mapped into a common subspace and a classifier f_s is constructed for the source project so that defect knowledge can be transferred from source to target project, which is defined as:

$$f_s(x_t) = w_t \cdot x_t \quad (14)$$

In the iterative optimization of the classifier, the hinge loss function commonly used for binary classification tasks is used. This optimization has the overall goal of minimizing the hinge loss and thus maximizing the margin between the training data points and the decision boundary. The hinge loss function is defined as follows:

$$l(x_t) = 1 - y_t w_t Z_t \quad (15)$$

where y_t is the true label, w_t is vector weight. Z_t represents the x_t data projected through the projection matrix V_t into the common subspace. The weight parameters of the model for the next iteration are updated as follows:

$$w_{t+1} = w_t + h_t y_t Z_t \quad (16)$$

$$h_t = \min \left\{ C, \frac{l(x_t)}{\|x_t\|_2} \right\} \quad (17)$$

where C is the aggressive parameter that controls the slack term affects on the objective function.

3.3 Online Stage

The COTL method, differing from most existing CP JIT-SDP methods, includes an online domain adaptation process to reduce the impact of concept drift on the target stream data distribution over time. Specifically, utilizing the projection matrix of target projects obtained in the offline stage, target online samples are transformed into the common subspace. Then, in the common subspace, classifiers are obtained for the online data corresponding to target projects. In order to predict the target online data, the source project classifiers and target project classifiers are fused into final classifiers, and weights are assigned to each classifier during the fusion process. Prediction labels are then generated for target online data. The loss is computed throughout the online process to update the fused classifiers.

In the online stage, classifier of source project trained in the offline stage is combined with online data classifier to form the final classifier (F_t), which can be expressed as follows:

$$F_t = \lambda_t f_s^t(Z_t) + \beta_t f_T^t(Z_t) \quad (18)$$

Where f_s^t is the classifier that was trained on the source project data with projection matrix mapped to the common subspace at round t , f_T^t is the classifier that was trained on the target project data with projection matrix mapped to the common subspace at round t . λ_t and β_t are

weights of classifiers of source and target project at round t , respectively, during the fusion process. The label of the prediction of online data in the target project is represented as:

$$\hat{y}_t = \text{sign}(F_t) \quad (19)$$

When the COTL method receives online data of newly submitted code change instances of the target project, it gets the relevant statistical features. COTL can map the features into the common subspace in real time using the projection matrix and use the final classifier to make predictions for the current submission instance. It also better adapts to the current situation of the target project by dynamically fine-tuning the weight parameters. The weight updating process for the classifier can be defined as follows:

$$\lambda_{t+1} = \frac{\lambda_t \cdot \delta_{q_t}}{\lambda_t \cdot \delta_{q_t} + \beta_t \cdot \delta_{r_t}} \quad (20)$$

$$\beta_{t+1} = \frac{\beta_t \cdot \delta_{r_t}}{\lambda_t \cdot \delta_{q_t} + \beta_t \cdot \delta_{r_t}} \quad (21)$$

Where $\delta \in (0, 1)$ is the weight discount parameter, q_t and r_t represent coefficients of the updating process.

In Section 3.2, we utilize labeled source project data and partially labeled target project data for cross-domain structure-preserving projection algorithm to reduce the distributional differences between the data. However, since information about the streaming data in target project is unknown, the partially labeled data in target project may not be consistent with the data distribution of future data, which leads to concept drift. To adapt the online data and minimize concept drift, we update the projection matrix of target project by reducing both marginal and conditional distribution differences between current target project data and offline target project data when receiving the t round of data. The calculation of the marginal distribution discrepancy (E_{mar}) is as follows:

$$E_{mar} = \left\| \frac{1}{n_{t_0}} \sum_{i=1}^{n_{t_0}} V_t x_{t_0} - \frac{1}{n_t} \sum_{i=1}^{n_t} V_t x_t \right\|_2^2 \quad (22)$$

where $\frac{1}{n_{t_0}} \sum_{i=1}^{n_{t_0}} V_t x_{t_0}$ denotes the mean of target project data during the training phase and $\frac{1}{n_t} \sum_{i=1}^{n_t} V_t x_t$ denotes the mean of target project data until round t . The calculation of the conditional distribution discrepancy (E_{con}) is as follows:

$$E_{con} = \left\| V_t \frac{1}{n_{t_0}} \left(\sum_{x_{t_0} \in D_{t_0}} x_{t_0} \right) - V_t \frac{1}{n_t} \left(\sum_{x_t \in D_t} x_t \right) \right\|_2^2 \quad (23)$$

where $\frac{1}{n_{t_0}} (\sum_{x_{t_0} \in D_{t_0}} x_{t_0})$ is the mean of the samples from each class in target project data in the training phase, $\frac{1}{n_t} (\sum_{x_t \in D_t} x_t)$ is the mean of the samples from each class in target project data until round t .

The objective of iterative optimization is to have the differences between the marginal and conditional distributions of data minimized with small adjustments to the projection matrix. The update formula for the projection matrix of the target project V_{t+1} for the next round is as follows:

$$V_{t+1} = \arg \min_V (E_{mar} + \gamma E_{con}) \quad (24)$$

where γ is the regularization parameter. Ideally, the distribution of data for the target project during the training phase can be further consistent with the distribution of target project data after the t round, thus reducing the occurrence of concept drift. This alignment enables the classifier to better identify defects, leading to improved classification performance.

ALGORITHM 1: COTL method

Input: Source project offline data X_s , partial target project offline data X_{t_0} and target project online data X_t ; Dimension of the subspace d ; In the offline stage set the initial weight parameters of the source and target project classifiers as $\lambda_i = \beta_i = \frac{1}{2}, i = 1, 2, \dots, n$; Hyperparameters α, C .
Output: Predicted results for target data are not labeled in the online data for the target project;

- 1: Learn the projections V_s and V_t by solving the generalized eigenvalue problem in Equation (10) using the labeled data X_s and X_{t_0} ;
- 2: train a source project classifier f_s based on $Z_s = V_s^T x_i^s$;
- 3: **for** $t = T_1, T_2, \dots, T_n$ **do**
- 4: Obtaining the online data $x_t \in X_t$ and label y_t of the target project
- 5: Mapping the online data x_t of the target project to the corresponding common subspace $Z_t = V_t^T x_t$
- 6: Predicting the label $\hat{y}_t$ using Equations (18) and (19)
- 7: Updating the weights of the classifier using Equations (20) and (21)
- 8: Calculating the loss function l_i involves the use of Equation (15)
- 9: **if** $l_i \leq 0$ **then**
- 10: Not updating the model
- 11: **else**
- 12: Updating the model parameters using Equations (16) and (17)
- 13: **end if**
- 14: **if** $t = T_w$ **then**
- 15: Updating the projection matrix V_{t+1} for the target project using Equation (24)
- 16: **end if**
- 17: **end for**

In order to balance the projection matrix and adaptability to new data, we use the parameter T_w to control how often the target projection matrix is updated based on the number of new online data received. In this study, T_w is set to 30. Specifically, after every 30 new online target data are received, an update of the target projection matrix is computed to further align it with the changing data distribution

In summary, the pseudocode for the proposed COTL method, combining the offline with online stages, is presented in Algorithm 1. Its approximate time complexity calculation mainly consists of offline and online stages. Assume that m is the dimension of the features. In the offline stage, the complexity of Equation (10) eigenvalue decomposition is $O(m^3)$, the complexity of Equation (11) and Equation (12) matrix multiplication is $O((n_s + n_t)m)$, and the complexity of source project classifier is $O(n_s \cdot m)$. In the online stage, the complexity of learning the model of the target project is $O(n_t \cdot m)$ and the complexity of updating the final model is $O(1)$. Thus, the overall time complexity is $O(m^3 + (n_s + n_t)m + 1)$.

4 EVALUATION

4.1 Research Questions

In order to evaluate the effectiveness of our proposed online CP JIT-SDP framework COTL, we analyze it in a detailed comparison with traditional offline methods and state-of-the-art online learning methods. Our research focuses on the dynamic nature of mobile application projects, where data arrives continuously over time, which requires our approach to be able to adapt to new information without the need for complete retraining. This situation emphasizes the importance of online in

practical applications, especially in the field of mobile application defect prediction, where timely and accurate identification of defects can significantly improve the quality and reliability of the application. In order to critically evaluate the capability and performance of the COTL method, we address the following research questions in our evaluation:

RQ1: Does the proposed COTL method improve CP JIT-SDP performance compared to offline methods when target project data arrives sequentially?

Motivation: Practical applications involve the continuous arrival of new target project data over time, constituting an online learning scenario. This study investigates whether the proposed COTL method leads to better predictive performance on mobile applications compared to offline methods. Demonstrating improved accuracy with online learning would highlight its importance for practical usage.

RQ2: How does the proposed COTL method compare to existing state-of-the-art online learning techniques for online CP JIT-SDP?

Motivation: Since COTL is based on online transfer learning, it is critical to benchmark against existing online transfer learning algorithms. Comparative evaluation verifies the performance between the proposed method and the existing method and confirms its contribution.

RQ3: What is the impact of the offline and online stages on the overall performance of the proposed COTL method?

Motivation: Given that the COTL method comprises distinct offline and online stages, addressing different aspects of the CP JIT-SDP problem, it is crucial to understand the individual contribution of each stage to the framework's effectiveness.

RQ4: How do the hyperparameters α and C affect the performance of COTL method?

Motivation: The role of different hyperparameters in the COTL method needs to be explored to assess their impact and sensitivity on the prediction performance. By systematically tuning the hyperparameters, changes in the performance metrics such as F-measure, Precision, Recall and G-mean are observed to determine the optimal parameter configuration.

4.2 Benchmark Dataset

To answer the above research questions, we utilize a benchmark dataset consisting of 15 open-source Android mobile applications, which is currently the standard dataset for CP JIT-SDP research, collected and built using the COMMIT GURU platform [30] by Catolino et al. [46]. This dataset encompasses a variety of apps, including but not limited to Firewall, Alfresco, and Sync, illustrating the diverse complexity of mobile application development. Table 2 summarizes key statistics of these apps, including number of lines of code (#LOC), total commit instances (#Instances), defective commit instances (#Buggy), clean commit instances (#Clean), and defect rate (%Ratio). The applications vary in scale from 9,506 to 275,637 lines of code, reflecting the real-world complexity of mobile application development. Each commit instance is described by 14 engineering features capturing properties related to code complexity, developer activity, code churn, and so on. These features have been identified in prior work [6, 43] as most useful for CP JIT-SDP in android mobile applications. See Table 3 for a concise overview of the features.

4.3 Experimental Settings

For the experimental setup, since the benchmark dataset was collected from the COMMIT GURU platform [30] in reverse chronological order, and in order to align with the chronological order of application development in the real world, the dataset was processed in reverse order so that the earliest submitted data was at the top of the dataset. Within this ordered dataset, an initial 30% was used for offline training to lay the foundation for the initial model of the COTL Method. The subsequent 70% of the data simulates the streaming nature of application development and serves

Table 2. The Statistical Information of 15 Applications

Project	#URL of Repository	#LOC	#Instances	#Buggy	#Clean	%Ratio
Firewall	https://github.com/ukanth/afwall	77,243	1,025	414	611	40.4%
Alfresco	https://github.com/Alfresco/alfresco-android-app	152,047	1,004	214	790	21.3%
Sync	https://github.com/mozilla-services/android-sync	275,637	209	62	147	29.7%
Wallpaper	https://github.com/olivergeith/android_wallpaperDesigner	35,917	588	94	494	16.0%
Keyboard	https://github.com/AnySoftKeyboard/AnySoftKeyboard	114,784	2,971	819	2,152	27.6%
Apg	https://github.com/thialfihar/apg	151,204	3,780	1,304	2,476	34.5%
Lottie	https://github.com/airbnb/lottie-android	57,291	235	36	199	15.3%
Kiwix	https://github.com/kiwix/kiwix-android	32,598	1,373	350	1,023	25.5%
Cloud	https://github.com/owncloud/android	115,169	3,700	830	2,870	22.4%
Turner	https://github.com/NightWhistler/PageTurner	30,943	164	23	141	14.0%
Reddit	https://github.com/emmaguy/wear-notify-for-reddit	9,506	222	60	162	27.0%
Image	https://github.com/nostra13/Android-Universal-Image-Loader	16,530	875	141	734	16.1%
Scroll	https://github.com/ksoichiro/Android-ObservableScrollView	27836	250	54	196	21.6%
Applozic	https://github.com/debugger22/Applozic-Android-SDK	87,662	946	143	803	15.1%
Delta	https://github.com/deltachat/deltachat-android	96971	2,465	185	2,280	7.5%

Table 3. Brief Descriptions of the 14 Features

Scope	Name	Definition
Diffusion	NS	Number of modified subsystems involved in the current change
	ND	Number of modified directories involved in the current change
	NF	Number of modified files involved in the current change
	Entropy	Distribution of modified code across files involved in the current change
Size	LA	Lines of code added by the current change
	LD	Lines of code deleted by the current change
	LT	Lines of code in a file before the current change
Purpose	FIX	Whether or not the current change is a bug fix
History	NDEV	Number of developers changing the files
	AGE	Average time interval since the last change
	NUC	Number of unique change to modified files
Experience	EXP	Developer experience
	REXP	Recent developer experience
	SEXP	Developer experience on a subsystem

as the target data for the online stage of testing. Additionally, in order to determine the optimal hyperparameters for the COTL method, a careful grid search process was employed, resulting in the selection of $\alpha = 10$, $C = 5$, $\gamma = 1$, and $T_w = 30$. This tuning is based on intenal validation procedures that assess the method’s performance across a range of parameter configurations, ensuring that the chosen parameters are ideally suited for maximizing the effectiveness of our method.

Our experiments are conducted in a controlled setting, utilizing an Intel(R) Core(TM) i5-11300H CPU operating at 3.10 GHz, with 16 GB of RAM, under a Windows 11 operating system.

4.4 Validation Strategy

To evaluate the model, we adopted the validation strategy from the CP-JIT-SDP study [6, 17, 43]. This method first selects one of the 15 open-source android mobile applications as the target project, and the remaining 14 projects are sequentially used as the source projects to form 210

Table 4. Confusion Matrix for Binary Classification Tasks

	Classified positive	Classified negative
Actual positive	TP	FN
Actual negative	FP	TN

cross-project task pairs. For each task pair, we repeat the experiment 20 times and calculate the average score to evaluate the model performance. This validation strategy helps to robustly test the performance of the COTL method in a cross-project setting.

4.5 Evaluation Metrics

When evaluating whether an instance of a code commit contains defects, typical binary classification task metrics can be used to compare the proposed method with the effectiveness of the comparison method. In binary classification analysis, confusion matrices are commonly used to measure the performance of the classifier. The confusion matrix (Table 4) divides each instance into **TP (True Positive)**, **FP (False Positive)**, **TN (True Negative)**, or **FN (False Negative)** based on the instance's true label and classification result. In this study, we employ four metrics to assess the performance of our COTL method, including F-measure, precision, recall, and G-mean. These metrics are commonly used in previous defect prediction [32, 45, 49, 51] and online transfer learning [48, 51] research. Among them, precision, recall, and F-measure can be calculated from the confusion matrix of classification results. These metrics are described in more detail as follows.

Precision: the fraction of positive predictions among all positive instances. A higher value is better. It is calculated using the formula:

$$Precision = \frac{TP}{TP + FP} \quad (25)$$

Recall: the fraction of positive instances that are correctly classified. A higher value is better. It is calculated using the formula:

$$Recall = \frac{TP}{TP + FN} \quad (26)$$

F-measure: the harmonic mean of precision and recall. A higher value is better. It is calculated using the formula:

$$F - measure = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (27)$$

G-mean: the geometric mean of recall for both positive and negative instances, accounting for balance. A higher value is better. It is calculated using the formula:

$$G - mean = \sqrt{\frac{TP \times TN}{(TP + FN)(TN + FP)}} \quad (28)$$

4.6 Statistic Test

To statistically compare two models, we utilized the Wilcoxon signed-rank test [38] and the Cliff's Delta test [7]. The Wilcoxon signed-rank test is widely adopted in defect prediction studies [22, 46] as it can assess differences without assuming an underlying data distribution. Unlike parametric tests, the Wilcoxon test does not require the data to follow a normal distribution. At 95% confidence, a p-value below 0.05 indicates a statistically significant difference between two models, while a p-value above 0.05 implies an insignificant difference. The Cliff's Delta test is a non-parametric effect size measure of the magnitude of the difference between the two methods. It calculates δ by comparing all possible pairwise combinations in two sets of data, with a value of δ between -1

Table 5. Standardized Cliff's Delta Test for Effect Levels

Cliff's Delta	level of effect size
$\ \delta\ < 0.147$	Negligible(N)
$0.147 \leq \ \delta\ < 0.330$	Small(S)
$0.330 \leq \ \delta\ < 0.474$	Medium(M)
$\ \delta\ \geq 0.474$	Large(L)

and 1. A larger absolute value of δ indicates more non-overlap between the two sets of data, i.e., a more significant difference between the two methods. Table 5 shows the level of difference in effect size by referring to the criteria of [47].

Additionally, in order to analyze the significance of our method and all comparative methods, we adopted a statistical test called **Scott-Knott effect size difference (Scott-Knott ESD)** [34]. It is a comparison method based on data means, which utilizes clustering analysis to group models with small differences into the same group. Then, based on the performance metrics of these models, it ranks them to visually demonstrate whether there are inter-group differences among different models and their rankings under evaluation metrics.

5 RESULTS ANALYSES

This section provides a wide range of experimental results as well as the results of the corresponding statistical analyses. Tables 6-13 illustrates the F-measure, Precision, Recall, and G-mean achieved by all comparison methods in various target projects. Larger values of the four metrics indicate better model performance. The specific calculation steps follow (as an example of F-measure calculation for **Transfer Component Analysis (TCA)**):

Step 1: One project from the list of 15 projects, as shown in the Table 2, is chosen as the target project (e.g., Firewall). Subsequently, each of the remaining 14 projects is employed as the source project for Firewall to create 14 distinct task pairs. The results for these task pairs are then calculated.

Step 2: The F-measure for TCA is determined as the average of the 14 results obtained in Step 1 when the target project is Firewall (i.e., The first row and second column of Table 6 has a value of 0.177).

Step 3: By analogy, use the same method to calculate the F-measure values for different online CP JIT-SDP methods under different target projects.

Step 4: Precision, Recall and G-mean are computed in a similar way as above steps.

The final average numerical results for the different methods in the 15 different target projects are listed in Tables 6-13 as an indicator of the final performance of each method. Each preceding row contains a bold value, representing the best-performing method within the same target project. The fourth-to-last row displays the degree of enhancement achieved by our method when contrasted with other methods. The third-to-last line illustrates how our method performs relative to other methods for **win/tie/loss (W/T/L)** outcomes. The second-to-last row presents the Wilcoxon signed rank test p-value for assessing the differences of our method relative to other methods. The last row presents the value of Cliff's Delta test, which is used to assess the difference in effect sizes between our method and the other methods.

5.1 RQ1: Does the Proposed COTL Method Improve CP JIT-SDP Performance Compared to Offline Methods when Target Project Data Arrives Sequentially?

Methods: To answer this question, we selected six offline CP JIT-SDP methods, **Transfer Component Analysis (TCA)** [28], **Joint Distribution based Transfer learning (JDT)** [23],

Balanced Distribution Adaptation-based transfer learning (BDA) [42], **Cross-triplet Deep Feature Embedding method (CDFE)** [43], **Kernel-based principal component analysis and Adversarial Learning technique methodology (KAL)** [6], and re-training offline method (BORB) [1] as baselines. TCA, JDT, and BDA were chosen because they are classical transfer learning techniques and are often used as benchmarks to evaluate transfer learning methods. In addition, CDFE and KAL, the latest techniques recently proposed specifically for CP JIT-SDP, were chosen to compare COTL with the current top-performing methods in the field. Due to the instability of online training in the face of concept drift, BORB, an offline training method that periodically starts from scratch, was also selected for comparison. Here, we provide a brief overview of these baseline methods.

TCA [28]: a method of early transfer learning that harmonizes the marginal distributions of source and target projects by dimensionality reduction techniques.

JDT [23]: a method that considers both marginal and conditional distributions of project data, but with equal weights.

BDA [42]: a method based on balanced distribution adaptation. It is similar to the JDT method, with the difference that it adaptively assigns different weights to the distribution.

CDFE [43]: a novel cross-domain feature embedding method using triplet loss. The method integrates a state-of-the-art triple loss function into a deep neural network to learn advanced feature representations of cross-application data, thereby reducing the differences between different application data.

KAL [6]: a method based on kernel function and adversarial learning techniques. The method utilizes kernel-based principal component analysis technique to capture representative features and employs adversarial learning to extract shared feature embeddings for model construction.

BORB [1]: a periodic offline training method from scratch that updates the training set using the most recently labeled instances, treating all the training instances received so far as a batch for retraining its predictive model from scratch based on an offline learning method.

Tables 6–9 present a comparison of the performance metrics of the COTL method we propose and six offline methods on the dataset, including F-measure, Precision, Recall, and G-mean. From these tables, the following conclusions can be drawn.

From Table 6, the best F-measure performance is achieved by our COTL method, which outperforms the six baseline methods, TCA, JDT, BDA, KAL, CDFE and BORB, in the 14 mobile applications. Compared to the baseline methods, our method shows average F-measure improvements of 132.052%, 67.257%, 55.569%, 121.062%, 39.651%, and 24.226%, respectively. From the perspective of wins/ties/losses (W/T/L), our COTL achieve success in almost every project, with only one loss in the KAL. From Table 7, our method outperformed the baseline methods in 11 applications, with the highest average precision reaching 0.493. Compared to the baseline, our average precision is improved by 46.658%, 57.606%, 46.302%, 16.350%, 65.202% and 45.309% across all applications. Our COTL achieve victories in 12–15 projects against each baseline, with only 0–3 losses. From Table 8, our method outperformed the baseline methods in 12 applications, with the highest average recall reaching 0.491. Compared to the baseline, our average recall is improved by 147.188%, 54.374%, 44.437%, 175.903%, 97.866% and 17.581% across all applications. Our COTL achieve victories in more than 12 projects against each baseline. From Table 9, our method outperformed the baseline methods in all applications, with the highest G-mean reaching 0.626. Compared to the baseline, our G-mean is improved by 129.569%, 29.123%, 23.650%, 87.389%, 69.636% and 14.116% across all applications. Our COTL achieve victories in all projects against each baseline.

Furthermore, from Table 6 to Table 9, it can be observed that our COTL method performs poorly in Wallpaper, Lottie, Turnner, Scroll, and Delta, indicating poor performance in some cases. For

Table 6. F-measure Comparison Results with Six Offline Methods

Target Project	TCA	JDT	BDA	CDFE	KAL	BORB	COTL
Firewall	0.177	0.323	0.367	0.151	0.327	0.523	0.571
Alfresco	0.218	0.294	0.310	0.157	0.307	0.430	0.498
Sync	0.257	0.310	0.352	0.231	0.407	0.374	0.474
Wallpaper	0.167	0.250	0.252	0.127	0.433	0.286	0.307
Keyboard	0.180	0.321	0.348	0.141	0.454	0.459	0.537
Apg	0.228	0.346	0.362	0.129	0.368	0.506	0.568
Lottie	0.195	0.238	0.257	0.229	0.296	0.292	0.347
Kiwix	0.196	0.306	0.311	0.132	0.313	0.409	0.546
Cloud	0.197	0.283	0.307	0.132	0.423	0.400	0.499
Turner	0.235	0.257	0.289	0.330	0.348	0.307	0.364
Reddit	0.358	0.406	0.419	0.358	0.494	0.502	0.662
Image	0.231	0.266	0.281	0.189	0.160	0.374	0.448
Scroll	0.169	0.282	0.307	0.216	0.321	0.353	0.421
Applozic	0.208	0.302	0.342	0.666	0.384	0.435	0.759
Delta	0.115	0.159	0.166	0.098	0.168	0.200	0.263
Average	0.209	0.290	0.311	0.219	0.347	0.390	0.484
Improvement	132.052%	67.257%	55.569%	121.062%	39.651%	24.226%	-
W/T/L	15/0/0	15/0/0	15/0/0	15/0/0	14/0/1	15/0/0	-
p-value	3.27E-4	3.27E-4	3.27E-4	3.27E-4	1.57E-3	3.27E-4	-
Cliff's Delta	0.973(L)	0.840(L)	0.751(L)	0.831(L)	0.591(L)	0.422(M)	-

Table 7. Precision Comparison Results with Six Offline Methods

Target Project	TCA	JDT	BDA	CDFE	KAL	BORB	COTL
Firewall	0.438	0.485	0.578	0.384	0.600	0.498	0.568
Alfresco	0.339	0.306	0.321	0.345	0.342	0.320	0.529
Sync	0.448	0.383	0.433	0.529	0.229	0.447	0.629
Wallpaper	0.241	0.210	0.218	0.305	0.194	0.181	0.323
Keyboard	0.312	0.346	0.382	0.332	0.333	0.378	0.519
Apg	0.533	0.452	0.459	0.338	0.399	0.460	0.605
Lottie	0.201	0.222	0.230	0.514	0.160	0.229	0.332
Kiwix	0.338	0.352	0.349	0.387	0.388	0.348	0.503
Cloud	0.264	0.293	0.334	0.340	0.314	0.324	0.464
Turner	0.222	0.217	0.244	0.317	0.130	0.247	0.368
Reddit	0.574	0.465	0.506	0.501	0.389	0.446	0.642
Image	0.336	0.270	0.262	0.440	0.285	0.387	0.452
Scroll	0.352	0.295	0.314	0.601	0.291	0.357	0.473
Applozic	0.349	0.283	0.310	0.649	0.262	0.334	0.739
Delta	0.096	0.110	0.113	0.373	0.158	0.133	0.247
Average	0.336	0.313	0.337	0.424	0.298	0.339	0.493
Improvement	46.658%	57.606%	46.302%	16.350%	65.202%	45.309%	-
W/T/L	15/0/0	15/0/0	14/0/1	12/0/3	14/0/1	15/0/0	-
p-value	3.27E-4	3.27E-4	4.03E-4	3.46E-2	4.03E-4	3.27E-4	-
Cliff's Delta	0.591(L)	0.707(L)	0.627(L)	0.298(S)	0.707(L)	0.636(L)	-

Table 8. Recall Comparison Results with Six Offline Methods

Target Project	TCA	JDT	BDA	CDFE	KAL	BORB	COTL
Firewall	0.122	0.265	0.285	0.101	0.364	0.476	0.576
Alfresco	0.198	0.309	0.327	0.114	0.273	0.364	0.475
Sync	0.220	0.293	0.305	0.154	0.253	0.376	0.401
Wallpaper	0.171	0.340	0.339	0.115	0.223	0.292	0.311
Keyboard	0.155	0.326	0.347	0.103	0.339	0.517	0.556
Apg	0.180	0.293	0.314	0.088	0.284	0.504	0.536
Lottie	0.226	0.282	0.329	0.197	0.181	0.330	0.392
Kiwix	0.167	0.300	0.302	0.102	0.237	0.457	0.599
Cloud	0.183	0.294	0.309	0.111	0.275	0.478	0.541
Turner	0.292	0.354	0.391	0.299	0.161	0.281	0.369
Reddit	0.315	0.391	0.392	0.259	0.369	0.640	0.685
Image	0.227	0.312	0.345	0.153	0.131	0.365	0.447
Scroll	0.152	0.332	0.347	0.171	0.239	0.329	0.398
Applozic	0.188	0.364	0.416	0.596	0.281	0.582	0.780
Delta	0.184	0.317	0.349	0.107	0.113	0.271	0.302
Average	0.199	0.318	0.340	0.178	0.248	0.418	0.491
Improvement	147.188%	54.374%	44.437%	175.903%	97.866%	17.581%	-
W/T/L	15/0/0	13/0/2	12/0/3	15/0/0	15/0/0	15/0/0	-
p-value	3.27E-4	8.93E-4	1.08E-3	3.27E-4	3.27E-4	3.27E-4	-
Cliff's Delta	0.982(L)	0.840(L)	0.716(L)	0.893(L)	0.938(L)	0.333(M)	-

Table 9. G-Mean Comparison Results with Six Offline Methods

Target Project	TCA	JDT	BDA	CDFE	KAL	BORB	COTL
Firewall	0.168	0.439	0.479	0.229	0.631	0.503	0.610
Alfresco	0.276	0.485	0.500	0.256	0.447	0.554	0.643
Sync	0.262	0.455	0.492	0.355	0.235	0.533	0.578
Wallpaper	0.306	0.491	0.490	0.253	0.319	0.441	0.512
Keyboard	0.234	0.480	0.504	0.232	0.409	0.621	0.647
Apg	0.241	0.477	0.490	0.219	0.480	0.573	0.647
Lottie	0.347	0.466	0.491	0.399	0.295	0.505	0.552
Kiwix	0.246	0.473	0.478	0.231	0.500	0.571	0.671
Cloud	0.276	0.467	0.487	0.233	0.408	0.618	0.642
Turner	0.376	0.504	0.540	0.472	0.181	0.537	0.566
Reddit	0.238	0.552	0.560	0.478	0.377	0.620	0.732
Image	0.307	0.486	0.511	0.321	0.451	0.491	0.631
Scroll	0.236	0.469	0.494	0.368	0.361	0.487	0.567
Applozic	0.256	0.534	0.575	0.725	0.341	0.655	0.867
Delta	0.321	0.489	0.500	0.237	0.345	0.517	0.523
Average	0.273	0.485	0.506	0.334	0.385	0.548	0.626
Improvement	129.569%	29.123%	23.650%	87.389%	69.636%	14.116%	-
W/T/L	15/0/0	15/0/0	15/0/0	15/0/0	14/0/1	15/0/0	-
p-value	3.27E-4	3.27E-4	3.27E-4	3.27E-4	3.27E-4	3.27E-4	-
Cliff's Delta	1.000(L)	0.964(L)	0.911(L)	0.884(L)	1.000(L)	0.547(L)	-

example, CDEF outperforms COTL in Precision metrics for Lottie, Scroll, and Delta. According to Table 2, this could be due to the fact that Wallpaper, Lottie, Turnner, and Delta have a low defect rate of about 15%, whereas Scroll, although it has a 21.6% defect rate, but the sample size is only 250. In offline methods like CDFE, they could utilize all the data from the target projects, enabling them to acquire more defect data and achieve better model performance. However, the COTL method achieves better performance in projects with higher defect rates and larger amounts of data that are closer to real-world scenarios such as Firewall, Apg and Keyboard. This is due to the fact that the COTL method receives the target project data in a streaming manner and has a higher likelihood of obtaining defective data at the initial stage, which allows the model to capture and adapt to the project features more efficiently during the online learning process, and thus exhibits better performance. The re-training BORB method outperforms the other offline methods in most metrics, suggesting that the model can be adapted to the features of the latest data through regular retraining cycles, thus remaining adaptive throughout the continuous training process. Nevertheless, the COTL method has advantages. It reduces distributional differences between projects and mitigates concept drift and adapts to new data during the online stage. COTL's ability to make real-time adjustments to model weights based on input data is particularly beneficial in the dynamically changing environment of mobile application data streams.

From the perspective of statistical hypothesis testing, in most cases, our COTL method and six offline CP JIT-SDP methods exhibit significant differences in the evaluation metrics F-measure, precision, recall, and G-mean. This can be determined by conducting a Wilcoxon signed-rank test with a p value of less than 0.05 at the 95% confidence level for paired samples. Therefore, it can be inferred that there are significant differences between our model and these comparative models. In the Cliff's Delta test, we observe that for the four evaluation metrics, the COTL method generally exhibits significant advantages over the baseline method, with large and medium effect sizes. In order to clearly demonstrate how these methods differ from COTL, we have plotted the box plot in Figure 4 based on the Scott-Knott ESD test results. From the test results, these methods have been ranked and grouped, with methods within the same color box exhibiting the least performance differences. From the graph, it can be observed that COTL ranks first in F-measure, precision, recall, and G-mean metrics, belonging to the highest level group. Furthermore, our method exhibits significant differences compared to the baseline methods, forming an independent group.

In summary, our COTL method differs from offline methods as it combines the advantages of online learning and transfer learning. It has the capability to simultaneously learn the features of both source and target projects and map them to a common subspace, thereby reducing the distribution difference between the two. Additionally, the classifier weights can be dynamically updated based on incoming new data, continuously improving the predictive performance of the model over time. Overall, the COTL method outperforms the offline method on all four metrics in the online CP JIT-SDP for mobile applications.

5.2 RQ2: How Does the Proposed COTL Method Compare to Existing State-of-the-Art Online Learning Techniques for Online CP JIT-SDP?

Methods: To answer this question, we selected seven online methods, PA [8], PAIO [8], HomOTL-I [48], HomTOL-II [48], HomTOL-K [48], HomOTLMS [39] and HomOTL-ODDM [9]. PA, PAIO, HomOTL-I, II and K were chosen because they are classical techniques for online learning to validate the advantages of COTL techniques over traditional methods. We also chose the recent techniques HomOTLMS and HomOTL-ODDM, which address the problem of distributional transfer in online scenarios. Considering that online training is unstable during the concept drift,

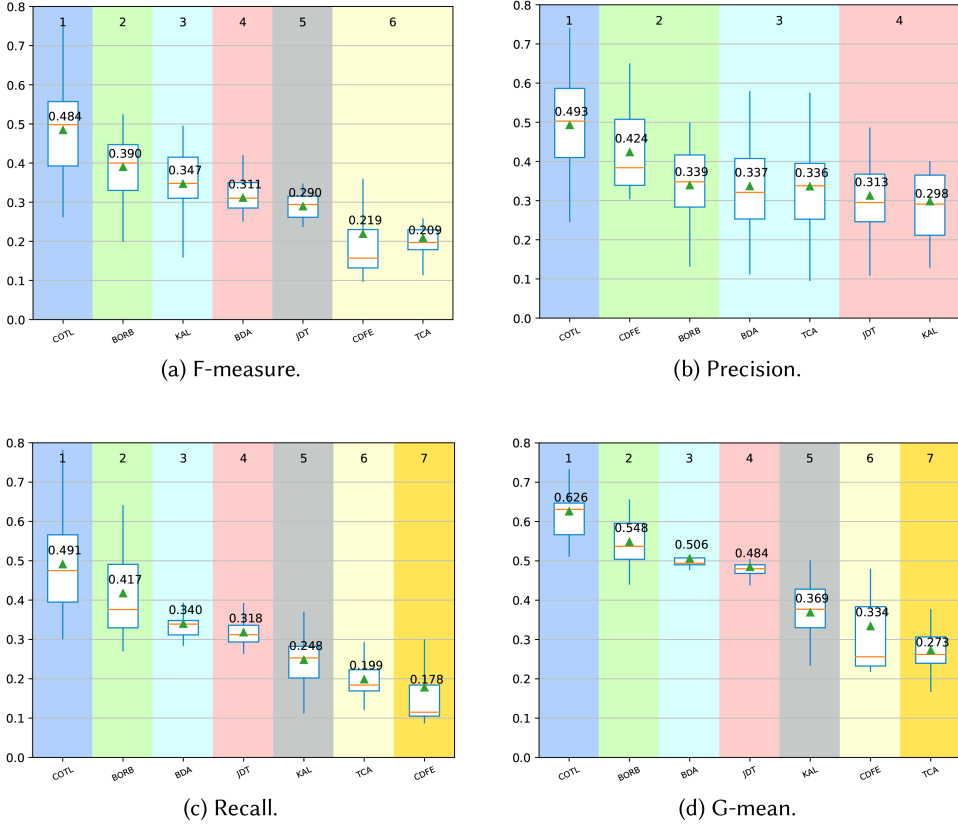


Fig. 4. Scott-Knott ESD rankings of different offline methods on four metrics.

we included the re-training BORB [1] to more fully evaluate the ability of COTL to mitigate concept drift. Here, we provide a brief overview of these baseline methods.

PA [8]: a classical online learning method that does not utilize any knowledge from the source domain and only updates the model parameters by minimizing a loss function.

PAIO [8]: a variant of the PA algorithm, which uses pre-trained classifier on the entire source domain to initialize the online learning process.

HomOTL-I [48]: a classical online transfer learning algorithm with a weight update strategy based on loss.

HomOTL-II [48]: a classical online transfer learning algorithm that employs an error-driven weight update strategy.

HomOTL-K [48]: a variant of the HomOTL-I algorithm that fixes the overall weight to 1/2 for the HomOTL-I algorithm, making it easier to verify the effectiveness of dynamically updating weights.

HomOTLMS [39]: a method by adaptively combining source and target classifiers which presents the idea of knowledge transfer using multiple source domains.

HomOTL-ODDM [9]: a method for reducing domain differences between source and target data during an online process.

BORB [1]: a periodic offline training method from scratch that updates the training set using the most recently labeled instances, treating all the training instances received so far as a batch for retraining its predictive model from scratch based on an offline learning method.

Table 10. F-Measure Comparison Results with Seven Online Methods and One Re-Training Method

Target Project	PA	PAIO	Hom OTL-I	Hom TOL-II	Hom TOL-K	Hom OTLMS	Hom OTL-ODDM	BORB	COTL
Firewall	0.524	0.544	0.165	0.278	0.518	0.573	0.523	0.523	0.571
Alfresco	0.407	0.490	0.161	0.165	0.407	0.463	0.311	0.430	0.498
Sync	0.432	0.435	0.153	0.104	0.400	0.199	0.254	0.374	0.474
Wallpaper	0.310	0.318	0.138	0.130	0.319	0.224	0.203	0.286	0.307
Keyboard	0.376	0.390	0.164	0.160	0.371	0.376	0.206	0.459	0.537
Apq	0.460	0.478	0.153	0.140	0.435	0.191	0.342	0.506	0.568
Lottie	0.274	0.290	0.098	0.089	0.296	0.157	0.195	0.292	0.347
Kiwix	0.379	0.433	0.100	0.099	0.363	0.313	0.395	0.409	0.546
Cloud	0.373	0.409	0.126	0.120	0.349	0.409	0.250	0.400	0.499
Turner	0.298	0.297	0.123	0.142	0.210	0.102	0.143	0.307	0.364
Reddit	0.353	0.490	0.202	0.202	0.352	0.312	0.508	0.502	0.662
Image	0.290	0.336	0.095	0.095	0.285	0.114	0.300	0.374	0.448
Scroll	0.315	0.346	0.042	0.038	0.292	0.181	0.210	0.353	0.421
Applozic	0.307	0.436	0.363	0.384	0.324	0.395	0.255	0.435	0.759
Delta	0.140	0.165	0.041	0.036	0.136	0.189	0.129	0.200	0.263
Average	0.349	0.391	0.142	0.146	0.337	0.280	0.281	0.390	0.484
Improvement	38.707%	23.995%	241.938%	232.799%	43.639%	73.964%	72.030%	24.226%	-
W/T/L	14/0/1	14/0/1	15/0/0	15/0/0	14/0/1	14/0/1	15/0/0	15/0/0	-
p-value	4.03E-4	4.94E-3	3.27E-4	3.27E-4	4.03E-4	4.03E-4	3.27E-4	3.27E-4	-
Cliff's Delta	0.582(L)	0.449(M)	0.973(L)	0.956(L)	0.627(L)	0.689(L)	0.760(L)	0.422(M)	-

Please note that the multiple source domain algorithm has been modified to a single source domain input in order to apply it to the online CP JIT-SDP task. All baseline methods were retrained under similar experimental settings to ensure fair comparisons.

Tables 10-13 present a comparison of the performance metrics, including F-measure, Precision, Recall, and G-mean, between our proposed method and eight baseline methods on the dataset. From these tables, the following conclusions can be drawn.

From Table 10, the best F-measure performance is achieved by our COTL method, which significantly outperforms the eight baseline methods in most mobile applications. Compared to the PA, PAIO, HomOTL-I, HomOTL-II, HomTOL-K, HomOTLMS, HomOTL-ODDM and BORB methods, our COTL method shows average F-measure improvements of 38.707%, 23.995%, 241.938%, 232.799%, 43.639%, 73.964%, 72.030% and 24.226%, respectively. From the perspective of wins/ties/losses (W/T/L), our COTL method achieves complete victories over most methods, only losing 1 times against PA, PAIO, HomTOL-K and HomOTLMS. From Table 11, our COTL method obtained significantly higher average precision values of 41.758%, 26.736%, 49.082%, 59.850%, 20.845%, 19.534%, 20.308% and 45.309%, respectively, when compared to the eight baseline methods. Our method achieves victories in 12-15 projects against each baseline method, with only two or fewer losses. From Table 12, our COTL method obtained significantly higher average recall values of 40.032%, 25.417%, 332.894%, 318.625%, 64.365%, 94.619%, 88.523% and 17.581%, respectively, when compared to the eight baseline methods. Compared to PA and PAIO, our COTL win 13/15 and 12/15 projects, respectively; against other baseline methods, our COTL achieve complete victories. From Table 13, our COTL method obtained significantly higher average G-mean values of 18.821%, 11.433%, 168.047%, 173.042%, 25.652%, 46.676%, 54.979% and 14.116%, respectively, when compared to the eight baseline methods. Our COTL achieve victories in 12-15 projects against each baseline, with only 0-3 losses.

Table 11. Precision Comparison Results with Seven Online Methods and One Re-Training Method

Target Project	PA	PAIO	Hom OTL-I	Hom TOL-II	Hom TOL-K	Hom OTLMS	Hom OTL-ODDM	BORB	COTL
Firewall	0.523	0.542	0.554	0.609	0.590	0.571	0.617	0.498	0.568
Alfresco	0.405	0.486	0.455	0.417	0.481	0.571	0.424	0.320	0.529
Sync	0.429	0.435	0.408	0.307	0.472	0.389	0.417	0.447	0.629
Wallpaper	0.309	0.320	0.209	0.155	0.385	0.421	0.428	0.181	0.323
Keyboard	0.376	0.388	0.301	0.281	0.459	0.396	0.347	0.378	0.519
Apg	0.459	0.476	0.530	0.350	0.507	0.598	0.552	0.460	0.605
Lottie	0.270	0.288	0.217	0.218	0.305	0.171	0.249	0.229	0.332
Kiwix	0.382	0.434	0.248	0.242	0.463	0.443	0.501	0.348	0.503
Cloud	0.372	0.407	0.255	0.242	0.421	0.402	0.380	0.324	0.464
Turner	0.292	0.290	0.239	0.183	0.250	0.119	0.149	0.247	0.368
Reddit	0.350	0.485	0.358	0.357	0.513	0.630	0.640	0.446	0.642
Image	0.289	0.336	0.335	0.390	0.364	0.414	0.540	0.387	0.452
Scroll	0.315	0.345	0.075	0.083	0.339	0.316	0.283	0.357	0.473
Applozic	0.306	0.432	0.656	0.708	0.386	0.502	0.374	0.334	0.739
Delta	0.141	0.168	0.119	0.084	0.181	0.241	0.243	0.133	0.247
Average	0.348	0.389	0.331	0.308	0.408	0.412	0.410	0.339	0.493
Improvement	41.758%	26.736%	49.082%	59.850%	20.845%	19.534%	20.308%	45.309%	-
W/T/L	15/0/0	15/0/0	15/0/0	14/0/1	13/0/2	12/0/3	12/0/3	15/0/0	-
p-value	3.27E-4	3.27E-4	3.27E-4	4.94E-4	1.57E-3	6.23E-3	1.34E-2	3.27E-4	-
Cliff's Delta	0.618(L)	0.467(M)	0.538(L)	0.627(L)	0.378(M)	0.307(S)	0.316(S)	0.636(L)	-

Table 12. Recall Comparison Results with Seven Online Methods and One Re-Training Method

Target Project	PA	PAIO	Hom OTL-I	Hom TOL-II	Hom TOL-K	Hom OTLMS	Hom OTL-ODDM	BORB	COTL
Firewall	0.524	0.547	0.467	0.111	0.189	0.575	0.481	0.476	0.576
Alfresco	0.410	0.494	0.366	0.134	0.139	0.417	0.285	0.364	0.475
Sync	0.435	0.435	0.362	0.121	0.076	0.157	0.237	0.376	0.401
Wallpaper	0.312	0.316	0.289	0.143	0.121	0.181	0.173	0.292	0.311
Keyboard	0.377	0.391	0.324	0.131	0.128	0.363	0.196	0.517	0.556
Apg	0.460	0.480	0.389	0.109	0.108	0.152	0.313	0.504	0.536
Lottie	0.278	0.292	0.260	0.083	0.081	0.165	0.182	0.330	0.392
Kiwix	0.375	0.433	0.306	0.079	0.081	0.280	0.381	0.457	0.599
Cloud	0.374	0.411	0.306	0.101	0.094	0.418	0.234	0.478	0.541
Turner	0.304	0.304	0.196	0.093	0.134	0.093	0.151	0.281	0.369
Reddit	0.356	0.496	0.299	0.169	0.176	0.219	0.483	0.640	0.685
Image	0.291	0.336	0.248	0.071	0.075	0.076	0.247	0.365	0.447
Scroll	0.315	0.348	0.267	0.034	0.032	0.161	0.197	0.329	0.398
Applozic	0.308	0.440	0.292	0.283	0.293	0.357	0.230	0.582	0.780
Delta	0.141	0.162	0.112	0.038	0.033	0.170	0.117	0.271	0.302
Average	0.351	0.392	0.299	0.113	0.117	0.252	0.260	0.418	0.491
Improvement	40.032%	25.417%	332.894%	318.625%	64.365%	94.619%	88.523%	17.581%	-
W/T/L	13/0/2	12/0/3	15/0/0	15/0/0	15/0/0	15/0/0	15/0/0	15/0/0	-
p-value	6.03E-4	1.31E-3	3.27E-4	3.27E-4	3.27E-4	3.27E-4	3.27E-4	3.27E-4	-
Cliff's Delta	0.609(L)	0.396(M)	1.000(L)	1.000(L)	0.840(L)	0.760(L)	0.813(L)	0.333(M)	-

Table 13. G-mean Comparison Results with Seven Online Methods and One Re-Training Method

Target Project	PA	PAIO	Hom OTL-I	Hom TOL-II	Hom TOL-K	Hom OTLMS	Hom OTL-ODDM	BORB	COTL
Firewall	0.596	0.614	0.274	0.406	0.600	0.638	0.586	0.503	0.610
Alfresco	0.586	0.652	0.286	0.286	0.566	0.604	0.443	0.554	0.643
Sync	0.573	0.576	0.254	0.158	0.539	0.339	0.354	0.533	0.578
Wallpaper	0.520	0.526	0.270	0.240	0.508	0.394	0.352	0.441	0.512
Keyboard	0.536	0.547	0.246	0.237	0.519	0.532	0.309	0.621	0.647
Apg	0.575	0.590	0.249	0.213	0.554	0.315	0.443	0.573	0.647
Lottie	0.490	0.503	0.200	0.182	0.485	0.346	0.340	0.505	0.552
Kiwix	0.545	0.591	0.171	0.171	0.514	0.463	0.529	0.571	0.671
Cloud	0.554	0.585	0.219	0.204	0.515	0.586	0.380	0.618	0.642
Turner	0.517	0.517	0.215	0.221	0.393	0.212	0.247	0.537	0.566
Reddit	0.520	0.633	0.267	0.260	0.499	0.444	0.593	0.620	0.732
Image	0.501	0.541	0.165	0.161	0.471	0.244	0.461	0.491	0.631
Scroll	0.505	0.533	0.076	0.071	0.472	0.327	0.316	0.487	0.567
Applozic	0.519	0.628	0.480	0.513	0.512	0.563	0.417	0.655	0.867
Delta	0.362	0.389	0.130	0.113	0.325	0.394	0.288	0.517	0.523
Average	0.527	0.562	0.233	0.229	0.498	0.427	0.404	0.548	0.626
Improvement	18.821%	11.433%	168.047%	173.042%	25.652%	46.676%	54.979%	14.116%	-
W/T/L	14/0/1	12/0/3	15/0/0	15/0/0	15/0/0	14/0/1	15/0/0	15/0/0	-
p-value	4.94E-4	1.89E-3	3.27E-4	3.27E-4	3.27E-4	4.03E-4	3.27E-4	3.27E-4	-
Cliff's Delta	0.707(L)	0.413(M)	1.000(L)	0.991(L)	0.831(L)	0.778(L)	0.876(L)	0.547(L)	-

Furthermore, from Tables 10 to 13, it can be observed that HomOTL-I, HomOTL-II, and HomTOL-K methods based on online transfer learning perform poorly in all four metrics. It is because these methods have not addressed the distributional differences that exist between source and target projects. On the other hand, HomOTLMS, HomOTL-ODDM, and COTL utilize different methods to handle distribution differences, resulting in superior performance compared to the aforementioned methods. For the BORB method that employs a re-training mechanism, it still outperforms most online learning methods on most metrics. This suggests that the problem of concept drift can be mitigated to some extent by periodically re-training the model with the latest data.

From the perspective of statistical hypothesis testing, in most cases, our COTL method and seven online learning methods and 1 re-training method exhibit significant differences in the evaluation metrics F-measure, precision, recall, and G-mean. This can be determined by conducting a Wilcoxon signed-rank test on paired samples, with a p-value less than 0.05 at a 95% confidence level. Therefore, it can be inferred that there are significant differences between our model and these comparative models. In the Cliff's Delta test, we observe that for the four evaluation metrics, the COTL method generally exhibits significant advantages over the baseline method, with large and medium effect sizes. In order to clearly demonstrate how these methods differ from COTL, we have plotted the box plot in Figure 5 based on the Scott-Knott ESD test results. From the test results, these methods have been ranked and grouped, with methods within the same color box exhibiting the least performance differences. From the graph, it can be observed that COTL ranks first in F-measure, precision, recall, and G-mean metrics, belonging to the highest level group. Furthermore, our method exhibits significant differences compared to the baseline methods, forming an independent group.

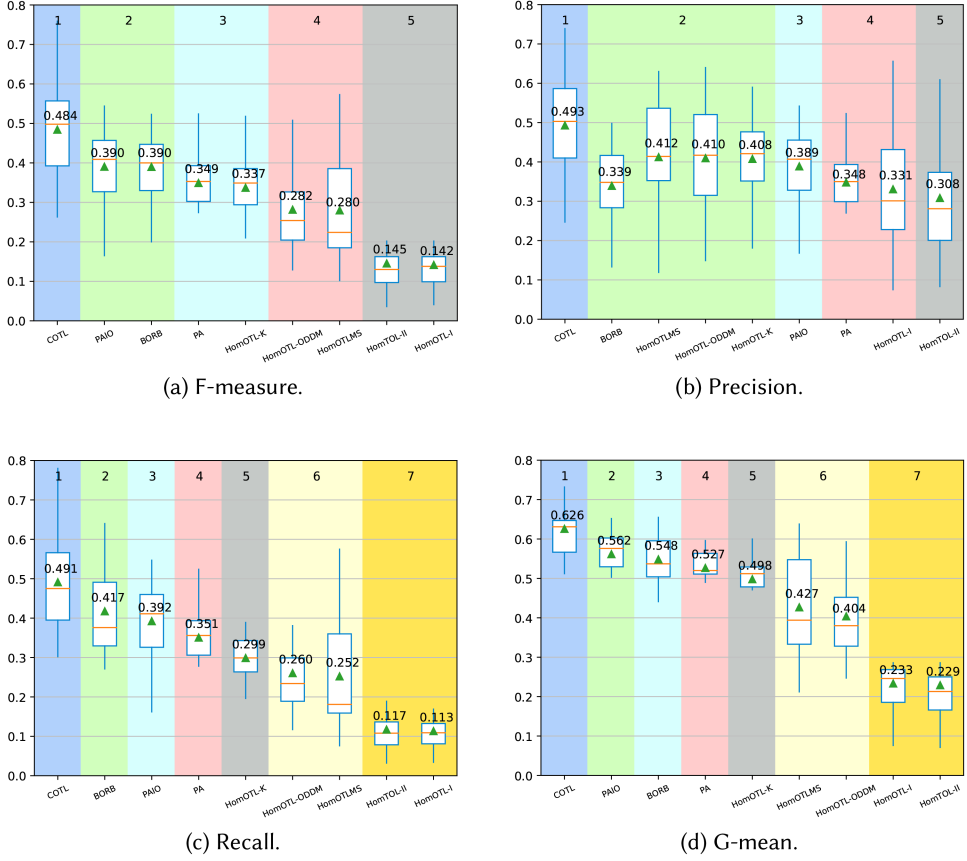


Fig. 5. Scott-Knott ESD rankings for different online and retraining methods on four metrics.

In summary, our COTL method builds on existing online transfer learning methods by incorporating the problem of concept drift in the target project stream data, while utilizing cross-domain structure-preserving projection to reduce the differences that exist between source and target projects. It addresses concept drift by simultaneously minimizing both marginal distribution discrepancy and conditional distribution discrepancy. Overall, our proposed COTL method outperforms state-of-the-art online learning methods and re-training method on all four metrics in online CP JIT-SDP for mobile applications.

5.3 RQ3: What is the Impact of the Offline and Online Stages on the Overall Performance of the Proposed COTL Method?

The experimental setup designed to answer RQ3 involved comparing the full COTL method with a variant that excludes the online stage (COTL-Offline). This comparison was essential to understand the contributions of the online stage to the method's overall performance in CP JIT-SDP. Given the inherent dependency of the online stage on the offline-generated projection matrix, conducting an experiment with only the online stage was not feasible. The absence of the online stage in the COTL-Offline experiment allowed us to indirectly assess the value added by the online adaptation.

The results in the Table 14 show that the full COTL framework improves the F-measure, Precision, Recall and G-mean metrics by 14.063%, 7.536%, 14.353% to 11.822% respectively when

Table 14. Results of Ablation Experiments on Four Evaluation Metrics

Metric	F-measure		Precision		Recall		G-mean	
Target Project	COTL-Offline	COTL	COTL-Offline	COTL	COTL-Offline	COTL	COTL-Offline	COTL
Firewall	0.545	0.571	0.537	0.568	0.481	0.576	0.582	0.610
Alfresco	0.458	0.498	0.497	0.529	0.285	0.475	0.599	0.643
Sync	0.372	0.474	0.608	0.629	0.237	0.401	0.487	0.578
Wallpaper	0.264	0.307	0.255	0.323	0.173	0.311	0.450	0.512
Keyboard	0.488	0.537	0.495	0.519	0.196	0.556	0.601	0.647
Apg	0.522	0.568	0.584	0.605	0.313	0.536	0.611	0.647
Lottie	0.277	0.347	0.297	0.332	0.182	0.392	0.456	0.552
Kiwix	0.527	0.546	0.482	0.503	0.381	0.599	0.647	0.671
Cloud	0.447	0.499	0.427	0.464	0.234	0.541	0.608	0.642
Turner	0.271	0.364	0.342	0.368	0.151	0.369	0.433	0.566
Reddit	0.539	0.662	0.578	0.642	0.483	0.685	0.631	0.732
Image	0.396	0.448	0.429	0.452	0.247	0.447	0.577	0.631
Scroll	0.397	0.421	0.450	0.473	0.197	0.398	0.542	0.567
Applozic	0.628	0.759	0.674	0.739	0.230	0.780	0.754	0.867
Delta	0.239	0.263	0.220	0.247	0.117	0.302	0.418	0.523
Average	0.425	0.484	0.458	0.493	0.260	0.491	0.560	0.626
Improvement	14.063%	-	7.536%	-	14.353%	-	11.822%	-
W/T/L	15/0/0	-	15/0/0	-	15/0/0	-	15/0/0	-
p-value	3.27E-4	-	3.24E-4	-	3.27E-4	-	3.27E-04	-
Cliff's Delta	0.353(M)	-	0.347(M)	-	0.362(M)	-	0.378(M)	-

compared to COTL-Offline. Across all 15 mobile applications, COTL outperforms COTL-Offline across all metrics. This result demonstrates the critical role of the online adaptation stage in improving the overall performance. In addition, the p-value of COTL and COTL-Offline is less than 0.05 under the Wilcoxon signed-rank test, indicating a significant difference between the two models. In the Cliff's Delta test, COTL exhibits advantages over COTL-Offline across four evaluation metrics, with an effect size classified as medium. The medium level of difference shown by Cliff's Delta is sufficient to affirm the effect of the online adaptation stage. It suggests that even with a medium effect size, the online stage contributes to the overall performance improvement. According to the Scott-Knott ESD test results in Figure 6, COTL ranks first in F-measure, precision, recall and G-mean metrics and forms a different group from COTL-Offline. This further demonstrates the effectiveness of the online stage.

5.4 RQ4: How Do the Hyperparameters α and C Affect the Performance of COTL Method?

We evaluated the sensitivity of the experimental results to hyperparameters, including two hyperparameters, denoted as α and C , as shown in Figure 7. Firstly, we investigated the impact of regularization parameter α in Equation (10) with α values of {0.01, 0.1, 1, 10, 100, 1000}. Figure 7(a) demonstrates that our COTL method performs better in F-measure, Precision, recall, and G-mean metrics when α is between 1 and 100, while smaller values of α lead to a decrease in all metrics. The results show the necessity of regularization in Equation (10). Therefore, in our experiments, we set α to 1. Figure 7(b) illustrates the experimental results of F-measure, Precision, recall, and G-mean metrics of our COTL method at different values of C . As C increases within the range of 1 to 5, all four metrics also increase, and they reach their maximum values when C is 5. The metrics

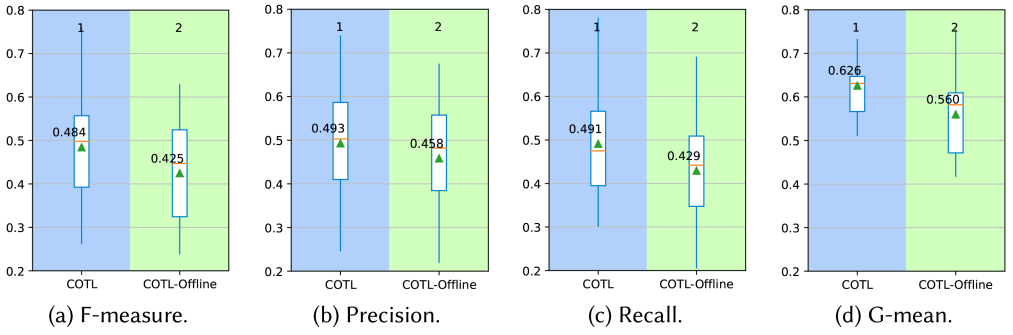


Fig. 6. Scott-Knott ESD rankings for ablation experiments on four metrics.

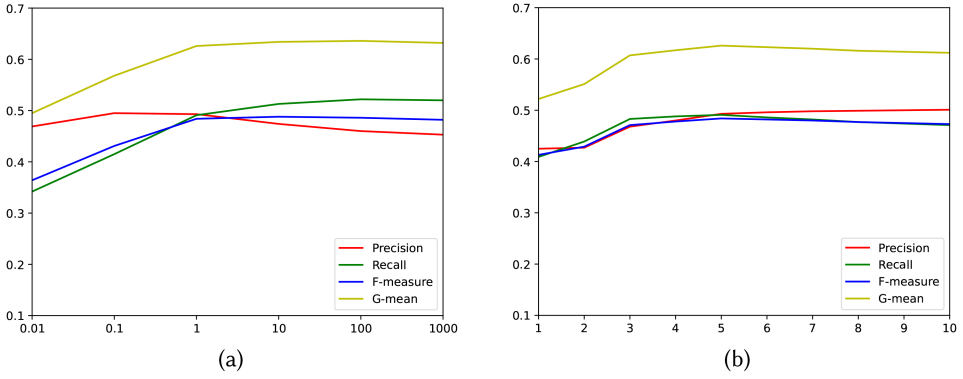


Fig. 7. Experimental results for four metrics with different parameters in the COTL method. (a) Results for parameter α . (b) Results for parameter C .

show little variation as C increased, indicating a stable trend. This suggests that larger values of C can improve the efficiency of transfer updates.

6 THREATS TO VALIDITY

6.1 Threats to Construct Validity

We have selected four widely used performance metrics for binary classification tasks as our evaluation criteria: F-measure, precision, recall, and G-mean. The metrics have already been recommended for evaluating the CP JIT-SDP task. However, they may not be the only suitable metrics. In binary classification tasks, there are other measurement methods that can be used, such as the **area under the ROC curve (AUC)** and the **Matthews correlation coefficient (MCC)**, for assessing performance. Moreover, to enhance the credibility of our findings, we have also employed Wilcoxon signed rank test and Scott-Knott ESD test that analyze the significant differences between our proposed method and the baseline methods.

6.2 Threats to Internal Validity

For re-implementing the baseline methods, we referred to source code where available or carefully followed the method descriptions in the papers. However, our re-implementations may not fully match the original implementations in all aspects. To minimize this threat, we validated our implementations on sample data before the experiments.

6.3 Threats to External Validity

We carefully selected a benchmark dataset containing 15 mobile applications to reduce bias. These applications have different scales and come from different application domains. However, these projects may not represent all software projects. Considering that the 15 mobile applications we selected are based on the android platform, there is a need to further explore the applicability of our method to applications based on other platforms in order to enhance the generalizability of our COTL method. In addition, these 15 mobile applications were developed using Java programming language. Further research is necessary to determine if our COTL method is also applicable to mobile applications developed in other programming languages.

7 CONCLUSION

This study proposed COTL, a novel online CP JIT-SDP framework for mobile applications. COTL addresses the key challenges of distribution discrepancy and concept drift in online CP JIT-SDP scenarios. In the offline stage, COTL utilizes cross-domain structure preserving projection to minimize distributional differences between source and target projects by mapping their data into a common subspace. This allows effective transfer the knowledge of defects from source to target projects. In the online stage, COTL leverages online transfer learning techniques to handle concept drift in the streaming target data. By reducing the differences in marginal and conditional distributions between offline and online data for target project, COTL adapts to changes in the target project over time. Comprehensive experiments were performed on 15 open-source mobile applications, comprising 210 cross-project pairs. The results compared to state-of-the-art offline and online learning baselines further demonstrate the superiority of our proposed COTL method. The online learning capability of COTL better equips it to deal with real-world challenges of evolving data. In order to facilitate the replication of our experimental results and to promote future research in this field of study, we have published the data, the baselines method and the implementation of the COTL method at <https://github.com/Me-HZH/COTL>.

Our work has the following implications for practice and future research:

- Based on the results of RQ1 and RQ2, the COTL framework enhances the ability of developers to detect and address defects during application development for practitioners looking to perform defect prediction. By leveraging cross-project data and online learning, developers can utilize historical defect data from other projects to predict new code changes in the pre-development phase, and update the model with continuously submitted code changes throughout development. This approach reduces the time and resources spent on debugging and testing, thereby increasing the efficiency of the development process. Implementing COTL also helps to improve the overall quality of mobile applications, ensuring a smoother and more reliable user experience by identifying and fixing defects early. This not only increases user satisfaction, but also strengthens market position by reducing the frequency of updates to address post-release issues.
- The results of RQ3 show that relying only on offline scenarios for defect prediction, without timely model updates for new data in a rapidly changing online application development environment, will result in lower prediction performance, which in turn increases the risk of missing defects and affects application quality.
- The COTL framework provides new ideas for exploring online JIT-SDP in mobile applications. By analyzing its effectiveness in different projects and development scenarios, researchers can further improve the defect prediction model.

Future work will increase the diversity of datasets to improve the generalizability of our method. We plan to incorporate more projects, including traditional software projects as well as programs

written in programming languages other than Java, to further validate the flexibility of our method in different software domains and development environments. Additionally, we will also explore a wider range of defect detection, especially by investigating multi-classification problems to identify different types of defects, thus enhancing the comprehensiveness of the predictions.

REFERENCES

- [1] George G. Cabral, Leandro Minku, Adriano LI Oliveira, Dinaldo Pessoa, and Sadia Tabassum. 2023. An investigation of online and offline learning models for online just-in-time software defect prediction. *Empirical Software Engineering* (2023). <https://doi.org/10.1007/s10664-023-10335-6>
- [2] George G. Cabral and Leandro L. Minku. 2022. Towards reliable online just-in-time software defect prediction. *IEEE Transactions on Software Engineering* 49, 3 (2022), 1342–1358. <https://doi.org/10.1109/TSE.2022.3175789>
- [3] George G. Cabral, Leandro L. Minku, Emad Shihab, and Suhaib Mujahid. 2019. Class imbalance evolution and verification latency in just-in-time software defect prediction. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 666–676. <https://doi.org/10.1109/ICSE.2019.00076>
- [4] Gemma Catolino. 2017. Just-in-time bug prediction in mobile applications: The domain matters!. In *2017 IEEE/ACM 4th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. IEEE, 201–202. <https://doi.org/10.1109/MOBILESoft.2017.58>
- [5] Gemma Catolino, Dario Di Nucci, and Filomena Ferrucci. 2019. Cross-project just-in-time bug prediction for mobile apps: An empirical assessment. In *2019 IEEE/ACM 6th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. IEEE, 99–110. <https://doi.org/10.1109/MOBILESoft.2019.00023>
- [6] Tian Cheng, Kunsong Zhao, Song Sun, Muhammad Mateen, and Junhao Wen. 2022. Effort-aware cross-project just-in-time defect prediction framework for mobile apps. *Frontiers of Computer Science* 16, 6 (2022), 166207. <https://doi.org/10.1007/s11704-021-1013-5>
- [7] Norman Cliff. 2014. *Ordinal Methods for Behavioral Data Analysis*. Psychology Press. <https://psycnet.apa.org/record/1997-97123-000>
- [8] Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. 2006. Online passive aggressive algorithms. 7, 19 (2006), 551–585. <http://jmlr.org/papers/v7/crammer06a.html>
- [9] Yuntao Du, Zhiwen Tan, Qian Chen, Yi Zhang, and Chongjun Wang. 2019. Homogeneous online transfer learning with online distribution discrepancy minimization. *arXiv preprint arXiv:1912.13226* (2019). <https://doi.org/10.48550/arXiv.1912.13226>
- [10] Norman E. Fenton and Martin Neil. 1999. A critique of software defect prediction models. *IEEE Transactions on Software Engineering* 25, 5 (1999), 675–689. <https://doi.org/10.1109/32.815326>
- [11] Xiaofei He and Partha Niyogi. 2003. Locality preserving projections. *Advances in Neural Information Processing Systems* 16 (2003). <https://dl.acm.org/doi/abs/10.5555/1144894>
- [12] Qiguo Huang, Zhengliang Li, and Qing Gu. 2023. Multi-task deep neural networks for just-in-time software defect prediction on mobile apps. *Concurrency and Computation: Practice and Experience* (2023), e7664. <https://doi.org/10.1002/cpe.7664>
- [13] Yue Jiang, Bojan Cukic, and Yan Ma. 2008. Techniques for evaluating fault prediction models. *Empirical Software Engineering* 13 (2008), 561–595. <https://doi.org/10.1007/s10664-008-9079-3>
- [14] Manzura Jorayeva, Akhan Akbulut, Cagatay Catal, and Alok Mishra. 2022. Deep learning-based defect prediction for mobile applications. *Sensors* 22, 13 (2022), 4734. <https://doi.org/10.3390/s22134734>
- [15] Manzura Jorayeva, Akhan Akbulut, Cagatay Catal, and Alok Mishra. 2022. Machine learning-based software defect prediction for mobile applications: A systematic literature review. *Sensors* 22, 7 (2022), 2551. <https://doi.org/10.3390/s22072551>
- [16] Marian Jureczko and Diomidis Spinellis. 2010. Using object-oriented design metrics to predict software defects. *Models and Methods of System Dependability. Oficyna Wydawnicza Politechniki Wrocławskiej* (2010), 69–81. <https://www2.dmst.aueb.gr/dds/pubs/conf/2010-DepCoS-RELCOMEX-ckjm-defects/html/JS10.pdf>
- [17] Yasutaka Kamei, Takafumi Fukushima, Shane McIntosh, Kazuhiro Yamashita, Naoyasu Ubayashi, and Ahmed E. Hassan. 2016. Studying just-in-time defect prediction using cross-project models. *Empirical Software Engineering* 21 (2016), 2072–2106. <https://doi.org/10.1007/s10664-015-9400-x>
- [18] Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2012. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering* 39, 6 (2012), 757–773. <https://doi.org/10.1109/TSE.2012.70>
- [19] Zhongfeng Kang, Bo Yang, Zesong Li, and Peng Wang. 2019. OTLAMC: An online transfer learning algorithm for multi-class classification. *Knowledge-Based Systems* 176 (2019), 133–146. <https://doi.org/10.1016/j.knosys.2019.03.024>

- [20] Zhongfeng Kang, Bo Yang, Shantian Yang, Xiaomei Fang, and Changjian Zhao. 2020. Online transfer learning with multiple source domains for multi-class classification. *Knowledge-Based Systems* 190 (2020), 105149. <https://doi.org/10.1016/j.knosys.2019.105149>
- [21] Xianglong Kong, Lingming Zhang, W. Eric Wong, and Bixin Li. 2015. Experience report: How do techniques, programs, and tests impact automated program repair?. In *2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 194–204. <https://doi.org/10.1109/ISSRE.2015.7381813>
- [22] Jian Li, Pinjia He, Jieming Zhu, and Michael R. Lyu. 2017. Software defect prediction via convolutional neural network. In *2017 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 318–328. <https://doi.org/10.1109/QRS.2017.42>
- [23] Mingsheng Long, Jianmin Wang, Guiguang Ding, Jianguang Sun, and Philip S. Yu. 2013. Transfer feature learning with joint distribution adaptation. In *Proceedings of the IEEE International Conference on Computer Vision*. 2200–2207. <https://doi.org/10.1109/iccv.2013.274>
- [24] William Martin, Federica Sarro, Yue Jia, Yuanyuan Zhang, and Mark Harman. 2016. A survey of app store analysis for software engineering. *IEEE Transactions on Software Engineering* 43, 9 (2016), 817–847. <https://doi.org/10.1109/TSE.2016.2630689>
- [25] Shane McIntosh and Yasutaka Kamei. 2018. Are fix-inducing changes a moving target? A longitudinal case study of just-in-time defect prediction. In *Proceedings of the 40th International Conference on Software Engineering*. 560–560. <https://doi.org/10.1145/3180155.3182514>
- [26] Tim Menzies, Jeremy Greenwald, and Art Frank. 2006. Data mining static code attributes to learn defect predictors. *IEEE Transactions on Software Engineering* 33, 1 (2006), 2–13. <https://doi.org/10.1109/TSE.2007.256941>
- [27] Chao Ni, Xin Xia, David Lo, Xiaohu Yang, and Ahmed E. Hassan. 2022. Just-in-time defect prediction on JavaScript projects: A replication study. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 4 (2022), 1–38. <https://doi.org/10.1145/3508479>
- [28] Sinno Jialin Pan, Ivor W. Tsang, James T. Kwok, and Qiang Yang. 2010. Domain adaptation via transfer component analysis. *IEEE Transactions on Neural Networks* 22, 2 (2010), 199–210. <https://doi.org/10.1109/TNN.2010.2091281>
- [29] Sinno Jialin Pan and Qiang Yang. 2010. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering* 22, 10 (2010), 1345–1359. <https://doi.org/10.1109/TKDE.2009.191>
- [30] Christoffer Rosen, Ben Grawi, and Emad Shihab. 2015. Commit guru: Analytics and risk prediction of software commits. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 966–969. <https://doi.org/10.1145/2786805.2803183>
- [31] Giuseppe Scanniello, Carmine Gravino, Andrian Marcus, and Tim Menzies. 2013. Class level fault prediction using software clustering. In *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 640–645. <https://doi.org/10.1109/ASE.2013.6693126>
- [32] Sadia Tabassum, Leandro L. Minku, and Danyi Feng. 2022. Cross-project online just-in-time software defect prediction. *IEEE Transactions on Software Engineering* 49, 1 (2022), 268–287. <https://doi.org/10.1109/TSE.2022.3150153>
- [33] Sadia Tabassum, Leandro L. Minku, Danyi Feng, George G. Cabral, and Liyan Song. 2020. An investigation of cross-project learning in online just-in-time software defect prediction. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 554–565. <https://doi.org/10.1145/3377811.3380403>
- [34] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2016. An empirical comparison of model validation techniques for defect prediction models. *IEEE Transactions on Software Engineering* 43, 1 (2016), 1–18. <https://doi.org/10.1109/TSE.2016.2584050>
- [35] Raymon van Dinter, Catayatal, Gökem Giray, and Bedir Tekinerdogan. 2023. Just-in-time defect prediction for mobile applications: Using shallow or deep learning? *Software Quality Journal* (2023), 1–22. <https://doi.org/10.1007/s11219-023-09629-1>
- [36] Lu Wang, Xiaobing Sun, Jingwei Wang, Yucong Duan, and Bin Li. 2017. Construct bug knowledge graph for bug resolution. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. IEEE, 189–191. <https://doi.org/10.1109/ICSE-C.2017.102>
- [37] Qian Wang and Toby P. Breckon. 2022. Cross-domain structure preserving projection for heterogeneous domain adaptation. *Pattern Recognition* 123 (2022), 108362. <https://doi.org/10.1016/j.patcog.2021.108362>
- [38] Frank Wilcoxon. 1992. Individual comparisons by ranking methods. In *Breakthroughs in Statistics: Methodology and Distribution*. Springer, 196–202. https://doi.org/10.1007/978-1-4612-4380-9_16
- [39] Qingyao Wu, Hanrui Wu, Xiaoming Zhou, Minghui Tan, Yonghui Xu, Yuguang Yan, and Tianyong Hao. 2017. Online transfer learning with multiple homogeneous or heterogeneous sources. *IEEE Transactions on Knowledge and Data Engineering* 29, 7 (2017), 1494–1507. <https://doi.org/10.1109/TKDE.2017.2685597>
- [40] Qingyao Wu, Xiaoming Zhou, Yuguang Yan, Hanrui Wu, and Huaqing Min. 2017. Online transfer learning by leveraging multiple source domains. *Knowledge and Information Systems* 52 (2017), 687–707. <https://doi.org/10.1007/s10115-016-1021-1>

- [41] Zhou Xu, Shuai Li, Jun Xu, Jin Liu, Xiapu Luo, Yifeng Zhang, Tao Zhang, Jacky Keung, and Yutian Tang. 2019. LDFR: Learning deep feature representation for software defect prediction. *Journal of Systems and Software* 158 (2019), 110402. <https://doi.org/10.1016/j.jss.2019.110402>
- [42] Zhou Xu, Shuai Pang, Tao Zhang, Xia-Pu Luo, Jin Liu, Yu-Tian Tang, Xiao Yu, and Lei Xue. 2019. Cross project defect prediction via balanced distribution adaptation based transfer learning. *Journal of Computer Science and Technology* 34 (2019), 1039–1062. <https://doi.org/10.1007/s11390-019-1959-z>
- [43] Zhou Xu, Kunsong Zhao, Tao Zhang, Chunlei Fu, Meng Yan, Zhiwen Xie, Xiaohong Zhang, and Gemma Catolino. 2021. Effort-aware just-in-time bug prediction for mobile apps via cross-triplet deep feature embedding. *IEEE Transactions on Reliability* 71, 1 (2021), 204–220. <https://doi.org/10.1109/TR.2021.3066170>
- [44] Xinli Yang, David Lo, Xin Xia, Yun Zhang, and Jianling Sun. 2015. Deep learning for just-in-time defect prediction. In *2015 IEEE International Conference on Software Quality, Reliability and Security*. IEEE, 17–26. <https://doi.org/10.1109/QRS.2015.14>
- [45] Zhengran Zeng, Yuqun Zhang, Haotian Zhang, and Lingming Zhang. 2021. Deep just-in-time defect prediction: How far are we?. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 427–438. <https://doi.org/10.1145/3460319.3464819>
- [46] Kunsong Zhao, Zhou Xu, Meng Yan, Lei Xue, Wei Li, and Gemma Catolino. 2022. A compositional model for effort-aware just-in-time defect prediction on Android apps. *IET Software* 16, 3 (2022), 259–278. <https://doi.org/10.1049/sfw2.12040>
- [47] Kunsong Zhao, Zhou Xu, Tao Zhang, Yutian Tang, and Meng Yan. 2021. Simplified deep forest model based just-in-time defect prediction for Android mobile apps. *IEEE Transactions on Reliability* 70, 2 (2021), 848–859. <https://doi.org/10.1109/TR.2021.3060937>
- [48] Peilin Zhao, Steven C. H. Hoi, Jiale Wang, and Bin Li. 2014. Online transfer learning. *Artificial Intelligence* 216 (2014), 76–102. <https://doi.org/10.1016/j.artint.2014.06.003>
- [49] Yunhua Zhao, Kostadin Damevski, and Hui Chen. 2023. A systematic survey of just-in-time software defect prediction. *Comput. Surveys* 55, 10 (2023), 1–35. <https://doi.org/10.1145/3567550>
- [50] Yuming Zhou, Yibiao Yang, Hongmin Lu, Lin Chen, Yanhui Li, Yangyang Zhao, Junyan Qian, and Baowen Xu. 2018. How far we have progressed in the journey? An examination of cross-project defect prediction. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 27, 1 (2018), 1–51. <https://doi.org/10.1145/3183339>
- [51] Kun Zhu, Nana Zhang, Shi Ying, and Dandan Zhu. 2020. Within-project and cross-project just-in-time defect prediction based on denoising autoencoder and convolutional neural network. *IET Software* 14, 3 (2020), 185–195. <https://doi.org/10.1049/iet-sen.2019.0278>

Received 20 November 2023; revised 6 March 2024; accepted 15 April 2024